УНИВЕРЗИТЕТ
У НОВОМ САДУ

ФАКУЛТЕТ
ТЕХНИЧКИХ НАУКА

Трг Доситеја Обрадовића 6, 21000 Нови Сад, Југославија
Деканат: 021 350-413; 021 450-810; Централа: 021 350-122
Рачуноводство: 021 58-220; Студентска служба: 021 350-763
Телефакс: 021 58-133; e-mail: ftndean@uns.ns.ac.yu

Сертификован
систем
квалитета

EN ISO 9001

PROJEKAT

Projektovanje složenih digitalnih sistema

TEMA PROJEKTA:

Prepoznavanje objekata na slici

MENTOR PROJEKTA:

Jana Janković

PROJEKAT IZARDIO:

Nikola Kovačić EE 77/2020
Ognjen Višnjić EE 217/2020

U Novom Sadu, 19.06.2025.

Sadržaj

1. Opis algoritma 3
2. Modifikacija algoritma i funkcionalnosti 5
3. ASM Dijagram 7
4. Datapath sistema 17
5. Dizajn i interfejsi 19
5.1 IP jezgro 19
5.1.1 Arhitektura modula 20
5.1.2 Tok rada modula 20
5.2 Memory subsystem 21
5.3 Dual port bram 22
5.4 Blok dijagram sistema 24
6. Analiza modelovanog sistem nakon sinteze i implementacije 24
6.1 Analiza utrošenih resursa 24
6.2 Analiza kritičnih putanja i maksimalne frekvencije rada sistema 25
6.3 Određivanje propusne moći (throughput) i kašnjenja (latency) sistema 25
6.4 Snaga 25
7. Pakovanje sistema u IP jezgro 27
8. Blok dizajn integrisanog sistema 28
9. Analiza integrisanog sistema nakon sinteze i implementacije i simulacije 28
9.1 Analiza utrošenih resursa 28
9.2 Analiza kritičnih putanja i maksimalne frekvencije rada sistema 29
9.3 Određivanje propusne moći (throughput) i kašnjenja (latency) sistema 29
9.4 Snaga 30
10. Zaključak 30

1. Opis algoritma

Algoritam je razvijen u programskom jeziku C++ uz korišćenje OpenCV biblioteke i omogućava detekciju i brojanje objekata (Pokemon likova) na osnovu njihove boje. Proces se sastoji iz više faza:

1. Učitavanje slike - Ulazna slika se učitava u boji. Ako se slika ne može učitati, algoritam se zaustavlja.
2. Segmentacija slike po boji (thresholding) - Za svaku boju koja predstavlja jednog Pokemona, definiše se donja i gornja granica u BGR prostoru. Pikseli čija boja spada u taj opseg označavaju se kao beli na binarnoj masci, dok su ostali crni.
3. Morfološka obrada (zatvaranje) - Da bi se eliminisali šumovi i spojile bliske regije iste boje, primenjuje se:
   - Dilacija: proširuje bele oblasti tako što uvećava svaku označenu tačku u kvadratnoj zoni 11×11.
   - Erozija: zatim se primenjuje erozija koja uklanja ivice koje nisu potpuno bele, zadržavajući samo kompaktne objekte.
4. Detekcija kontura pomoću DFS (Depth-First Search) algoritma - Umesto ugrađene OpenCV funkcije za pronalaženje kontura, koristi se ručno implementirana dubinska pretraga (DFS), koja obilazi susedne bele piksele i formira konture.
5. Filtriranje kontura - Za svaku pronađenu konturu izračunava se površina i širina bounding box-a. Objekti se filtriraju tako da površina mora biti u opsegu od 80 do 6050 piksela, a širina objekta mora biti manja od 300 piksela.
6. Označavanje i brojanje objekata - Svaki validan objekat se broji i broj prepoznatih objekata za svaku klasu se beleži i prikazuje u terminalu.

```cpp
void processColorContours(const cv::Mat& img, const cv::Scalar& lower, const cv::Scalar& upper, const cv::Scalar& color,
                          const std::string& label, int& objects_count, cv::Mat& marked_image) {
    // Step 1: Color thresholding
    cv::Mat mask = cv::Mat::zeros(img.size(), CV_8UC1);
    for (int y = 0; y < img.rows; ++y) {
        for (int x = 0; x < img.cols; ++x) {
            cv::Scalar pixel = img.at<cv::Vec3b>(y, x);
            if (pixel[0] >= lower[0] && pixel[1] >= lower[1] && pixel[2] >= lower[2] &&
                pixel[0] <= upper[0] && pixel[1] <= upper[1] && pixel[2] <= upper[2]) {
                mask.at<uchar>(y, x) = 255;
            }
        }
    }

    // Alternative Morphological Closing Operation
    cv::Mat closed_mask = mask.clone();  // Clone the original mask

    // Dilation operation (expanding bright regions)
    for (int y = 0; y < closed_mask.rows; ++y) {
        for (int x = 0; x < closed_mask.cols; ++x) {
            if (mask.at<uchar>(y, x) == 255) {
                // Set pixels in the neighborhood to 255
                for (int i = -5; i <= 5; ++i) {
                    for (int j = -5; j <= 5; ++j) {
                        int newY = y + i;
                        int newX = x + j;
                        if (newY >= 0 && newY < closed_mask.rows && newX >= 0 && newX < closed_mask.cols) {
                            closed_mask.at<uchar>(newY, newX) = 255;
                        }
                    }
                }
            }
        }
    }
```

```
// Erosion operation (shrinking bright regions)
cv::Mat temp_mask = closed_mask.clone();  // Create a temporary mask to avoid modifying pixels during erosion
for (int y = 0; y < closed_mask.rows; ++y) {
    for (int x = 0; x < closed_mask.cols; ++x) {
        if (closed_mask.at<uchar>(y, x) == 255) {
            // Check if all pixels in the neighborhood are 255
            bool allWhite = true;
            for (int i = -5; i <= 5; ++i) {
                for (int j = -5; j <= 5; ++j) {
                    int newY = y + i;
                    int newX = x + j;
                    if (newY >= 0 && newY < closed_mask.rows && newX >= 0 && newX < closed_mask.cols) {
                        if (temp_mask.at<uchar>(newY, newX) != 255) {
                            allWhite = false;
                            break;
                        }
                    }
                }
                if (!allWhite) {
                    break;
                }
            }
            // Set the pixel to 255 only if all pixels in the neighborhood are 255
            closed_mask.at<uchar>(y, x) = allWhite ? 255 : 0;
        }
    }
}

// Step 3: Find contours in the closed mask manually
d::vector<std::vector<cv::Point>> contours;
contours.clear();  // Clear the existing contours

cv::Mat visited = cv::Mat::zeros(closed_mask.size(), CV_8U);

static const int dx[] = {1, 1, 0, -1, -1, -1, 0, 1};
static const int dy[] = {0, 1, 1, 1, 0, -1, -1, -1};

for (int y = 0; y < closed_mask.rows; ++y) {
    for (int x = 0; x < closed_mask.cols; ++x) {
        if (closed_mask.at<uchar>(y, x) == 255 && visited.at<uchar>(y, x) == 0) {
            std::vector<cv::Point> contour;
            std::stack<cv::Point> stack;
            stack.push(cv::Point(x, y));

            while (!stack.empty()) {
                cv::Point current = stack.top();
                stack.pop();

                if (visited.at<uchar>(current.y, current.x) == 0 && closed_mask.at<uchar>(current.y, current.x) == 255) {
                    visited.at<uchar>(current.y, current.x) = 255;
                    contour.push_back(current);

                    for (int i = 0; i < 8; ++i) {
                        int newX = current.x + dx[i];
                        int newY = current.y + dy[i];

                        if (newX >= 0 && newX < closed_mask.cols && newY >= 0 && newY < closed_mask.rows) {
                            stack.push(cv::Point(newX, newY));
                        }
                    }
                }
            }
            contours.push_back(contour);
        }
    }
}

// Step 4: Parameters for object filtering
int min_object_area = 120;
int max_object_area = 6050;
int max_object_width = 300;

// Step 5: Process each contour
for (const auto& contour : contours) {
    // Calculate contour area and bounding rectangle
    double area = cv::contourArea(contour);
    cv::Rect rect = cv::boundingRect(contour);

    // Step 6: Object filtering based on area and width
    if (area > min_object_area && area < max_object_area && rect.width < max_object_width) {
        // Increment object count
        objects_count++;
        // Step 7: Draw rectangle around the object
        cv::rectangle(marked_image, rect, color, 2);
        // Step 8: Add label with object count
        cv::putText(marked_image, label + " " + std::to_string(objects_count), cv::Point(rect.x, rect.y - 10), cv::FONT_HERSHEY_SIMPLEX, 0.5, color, 2);
    }
}
```

Slika 1, Algoritam za preopoznavanje objekata

2. Modifikacija algoritma i funkcionalnosti

Modifikacija rada algoritma je napravljena u fazi modelovanja dizajna na RT nivou. U ovoj verziji algoritma, implementacija je izvedena korišćenjem eksplicitnih if i goto naredbi umesto standardnih for i while petlji. Ovaj pristup približava izvršavanje logici RTL (Register Transfer Level), gde se kontrola toka precizno upravlja bez implicitnog korišćenja viših programskih apstrakcija. Time se postiže finija kontrola nad svakim korakom izvršavanja, što je korisno za optimizaciju na mikrokontrolerskim arhitekturama i prilikom generisanja koda bliskog hardverskoj realizaciji.

```
int y = 0;
int x = 0;
// Create the mask
mask_outer:
    if (y >= rows) goto end_create_mask_outer;
    x = 0;
mask_inner:
    if (x >= cols) goto end_create_mask_inner;
    const unsigned char* pixel = img->data + (y * cols + x) * channels;
    if (pixel[0] >= lower[0] && pixel[1] >= lower[1] && pixel[2] >= lower[2] &&
        pixel[0] <= upper[0] && pixel[1] <= upper[1] && pixel[2] <= upper[2]) {
        mask[y * cols + x] = 255;
    }
    x++;
    goto mask_inner;
end_create_mask_inner:
    y++;
    goto mask_outer;
end_create_mask_outer:

// Simple dilation to fill small holes
memcpy(closed_mask, mask, rows * cols * sizeof(unsigned char));
y = 0;
x = 0;
dilation_outer:
    if (y >= rows) goto end_dilation_outer;
    x = 0;
dilation_inner:
    if (x >= cols) goto end_dilation_inner;
    if (mask[y * cols + x] == 255) {
        int i = -5;
    dilation_i:
        if (i > 5) goto end_dilation_i;
        int j = -5;
    dilation_j:
        if (j > 5) goto end_dilation_j;
        int newY = y + i;
        int newX = x + j;
        if (newY >= 0 && newY < rows && newX >= 0 && newX < cols) {
            closed_mask[newY * cols + newX] = 255;
        }
        j++;
        goto dilation_j;
    end_dilation_j:
        i++;
        goto dilation_i;
    end_dilation_i:
```

```
        }
        x++;
        goto dilation_inner;
    end_dilation_inner:
        y++;
        goto dilation_outer;
    end_dilation_outer:
    memcpy(temp_mask, closed_mask, rows * cols * sizeof(unsigned char));
    y = 0;
    x = 0;
    check_temp_pixel_outer:
        if (y >= rows) goto end_check_temp_pixel_outer;
        x = 0;
    check_temp_pixel_inner:
        if (x >= cols) goto end_check_temp_pixel_inner;
        if (closed_mask[y * cols + x] == 255) {
            int allWhite = 1;
            int i = -5;
        check_temp_expand:
            if (i > 5) goto end_check_temp_expand;
            int j = -5;
        check_temp_j:
            if (j > 5) goto end_check_temp_j;
            int newY = y + i;
            int newX = x + j;
            if (newY >= 0 && newY < rows && newX >= 0 && newX < cols) {
                if (temp_mask[newY * cols + newX] != 255) {
                    allWhite = 0;
                    goto end_check_temp_j;
                }
            }
            j++;
            goto check_temp_j;
        end_check_temp_j:
            i++;
            goto check_temp_expand;
        end_check_temp_expand:
            closed_mask[y * cols + x] = allWhite ? 255 : 0;
        }
        x++;
        goto check_temp_pixel_inner;
    end_check_temp_pixel_inner:
        y++;
        goto check_temp_pixel_outer;
    end_check_temp_pixel_outer:
    int stack_size = 0;
    y = 0;
    find_contours_outer:
        if (y >= rows) goto end_find_contours_outer;
        x = 0;
    find_contours_inner:
        if (x >= cols) goto end_find_contours_inner;
        if (closed_mask[y * cols + x] == 255 && visited[y * cols + x] == 0) {
            int area = 0;
            int min_x = cols, max_x = 0, min_y = rows, max_y = 0;
            stack[stack_size++] = (Point){x, y};
        find_contours_while:
            if (stack_size <= 0) goto end_find_contours_while;
            Point current = stack[--stack_size];
            int cx = current.x;
            int cy = current.y;
            if (visited[cy * cols + cx] == 0 && closed_mask[cy * cols + cx] == 255) {
                visited[cy * cols + cx] = 255;
                area++;
                if (cx < min_x) min_x = cx;
                if (cx > max_x) max_x = cx;
                if (cy < min_y) min_y = cy;
                if (cy > max_y) max_y = cy;
                int i = 0;
            find_contours_while_i:
                if (i >= 8) goto end_find_contours_while_i;
                int newX = cx + dx[i];
                int newY = cy + dy[i];
                if (newX >= 0 && newX < cols && newY >= 0 && newY < rows) {
                    stack[stack_size++] = (Point){newX, newY};
                }
                i++;
                goto find_contours_while_i;
            end_find_contours_while_i:
                ;
            }
            goto find_contours_while;
        end_find_contours_while:
        int width = max_x - min_x + 1;
        if (area > min_object_area && area < max_object_area && width < max_object_width) {
            (*objects_count)++;
        }
        }
}
```

Slika 2, Modifikovani algoritma

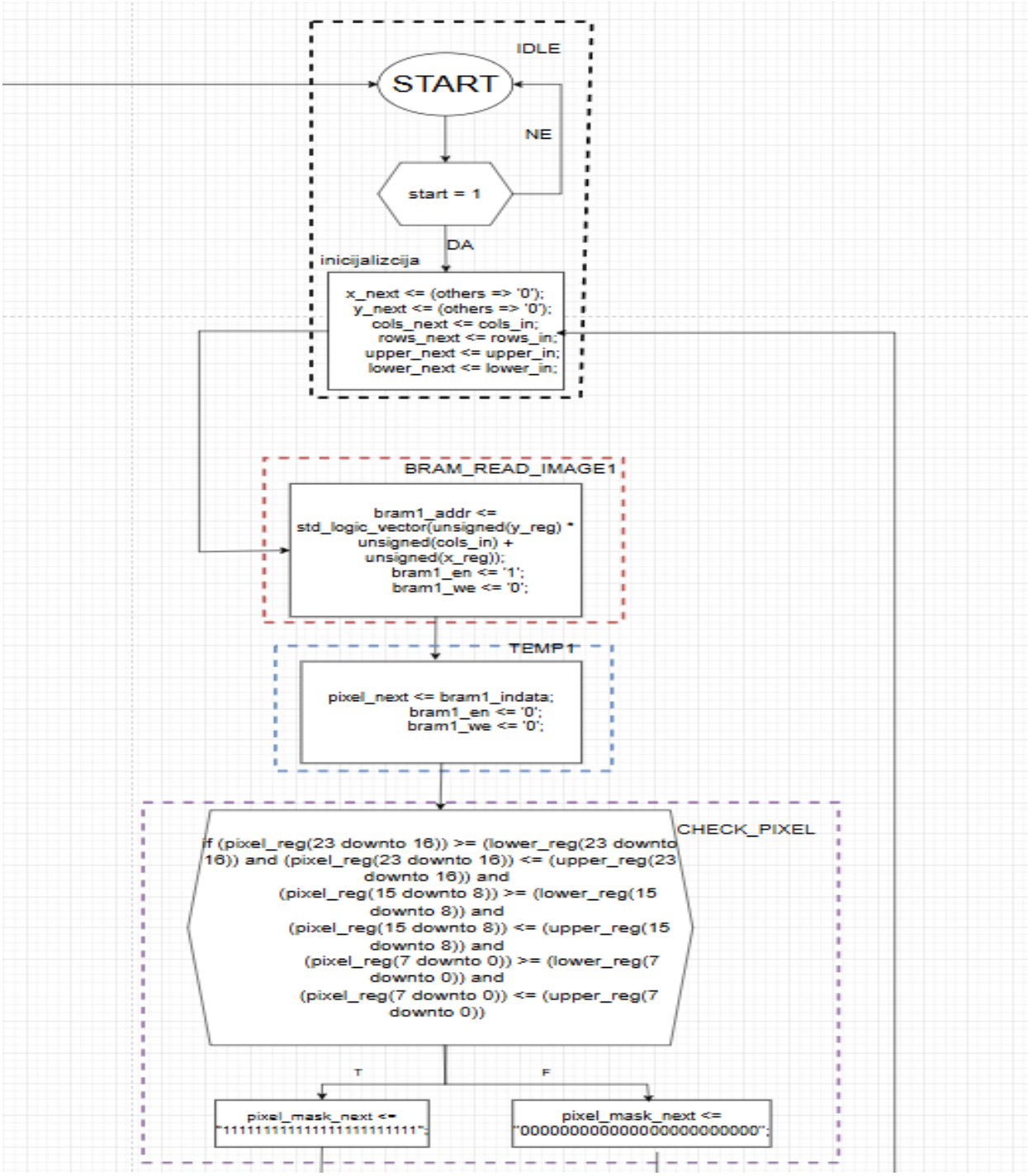
3. ASM Dijagram

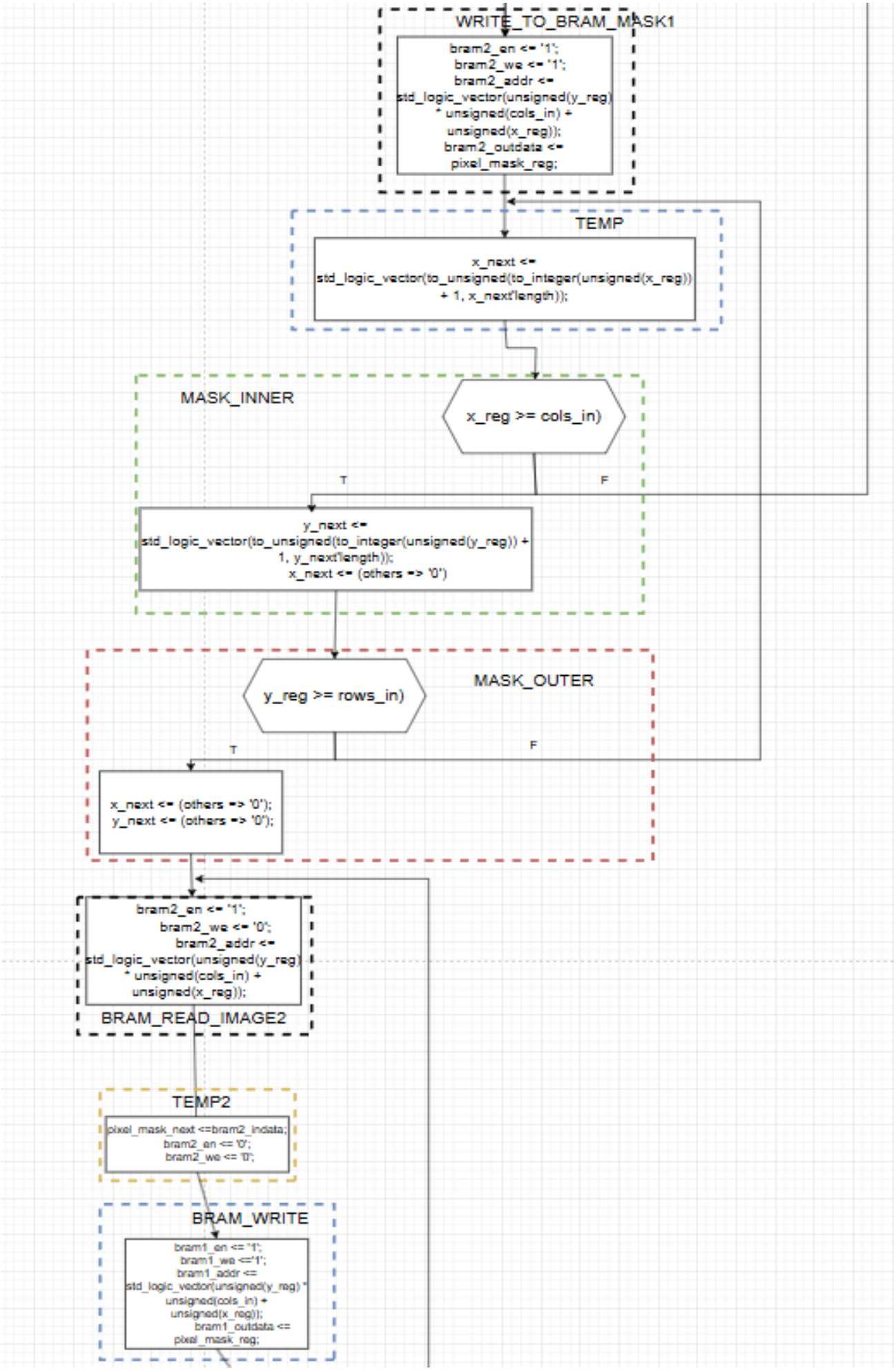
Nakon prevođenja originalnog algoritma u oblik sa eksplicitnim if i goto naredbama (RTL - podoban pseudokod), izrađen je ASM (Algorithmic State Machine) dijagram koji prikazuje tok izvršavanja na nivou stanja i prelaza. Ovaj dijagram koristi pristup da bi se algoritam precizno modelovao u terminima kontrolne logike, čime se omogućava direktno prevođenje u VHDL opis na Register Transfer Level (RT nivo).

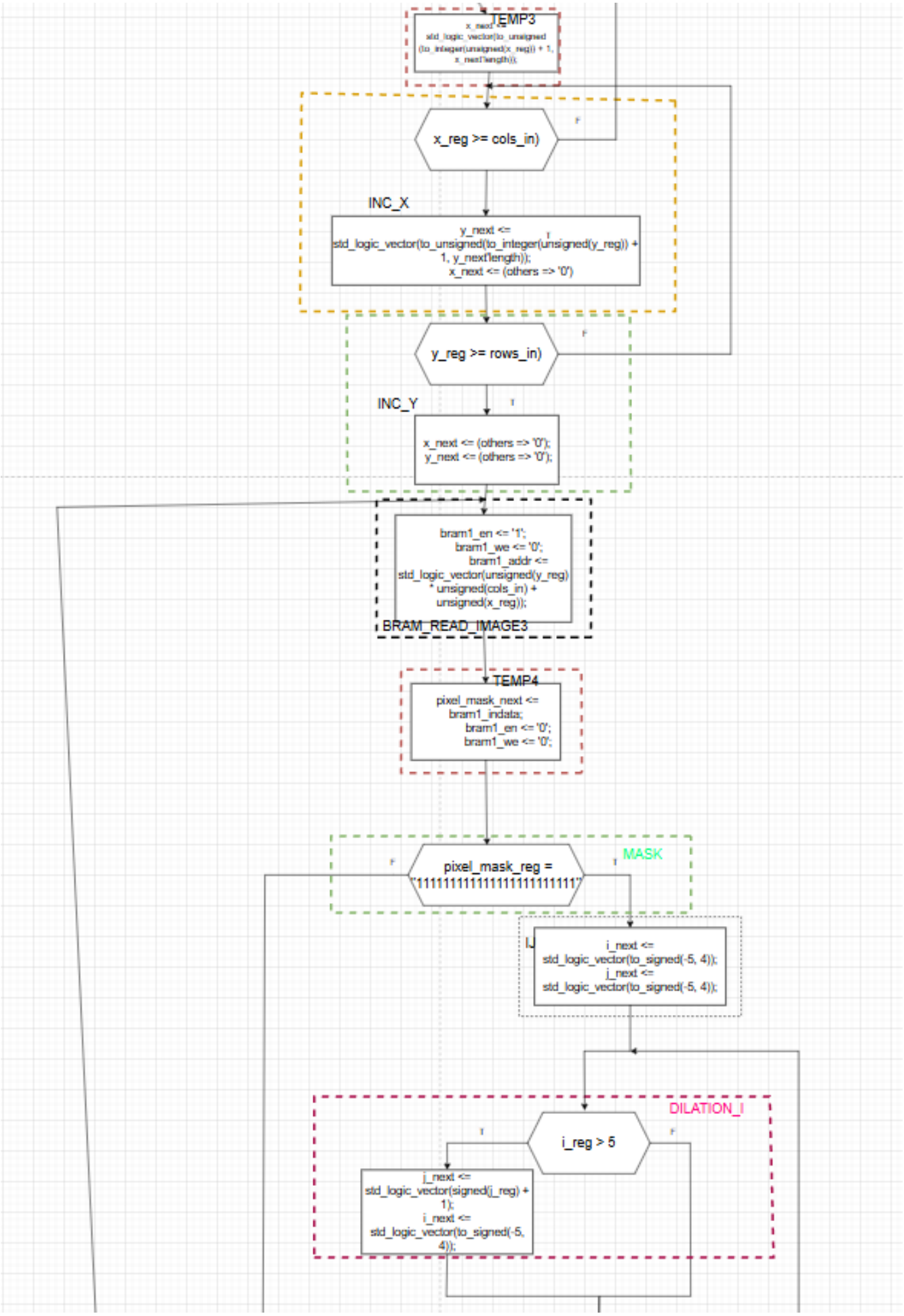
ASM dijagram definiše:

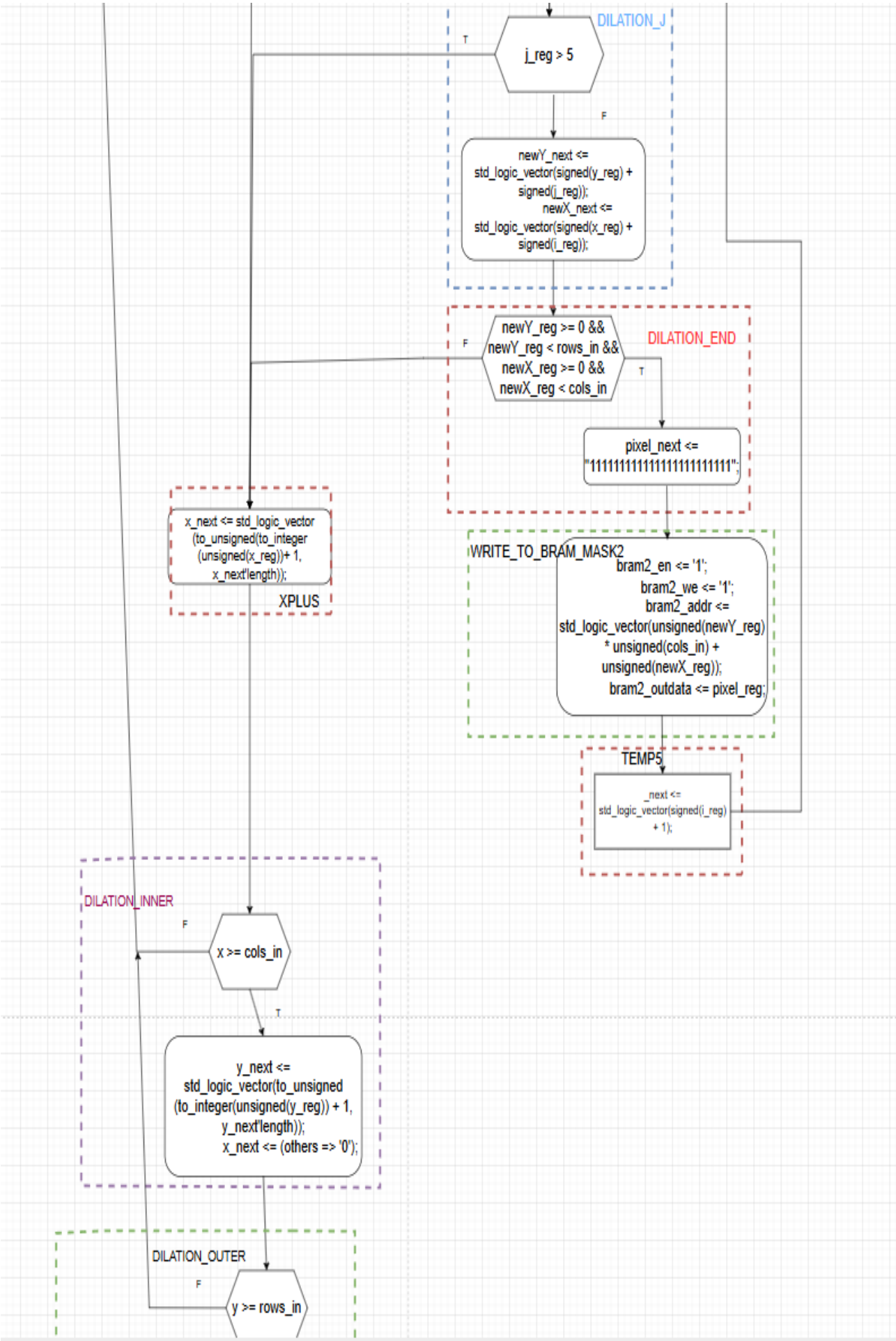
- Stanja za svaku fazu obrade (npr. detekcija boje, dilatacija, analiza maski, DFS pretraga kontura),
- Uslove prelaza između stanja na osnovu ulaza i internih signala,
- Akcije koje se izvršavaju u svakom stanju (npr. upis u memoriju, inkrement brojača, upis koordinata piksela itd).

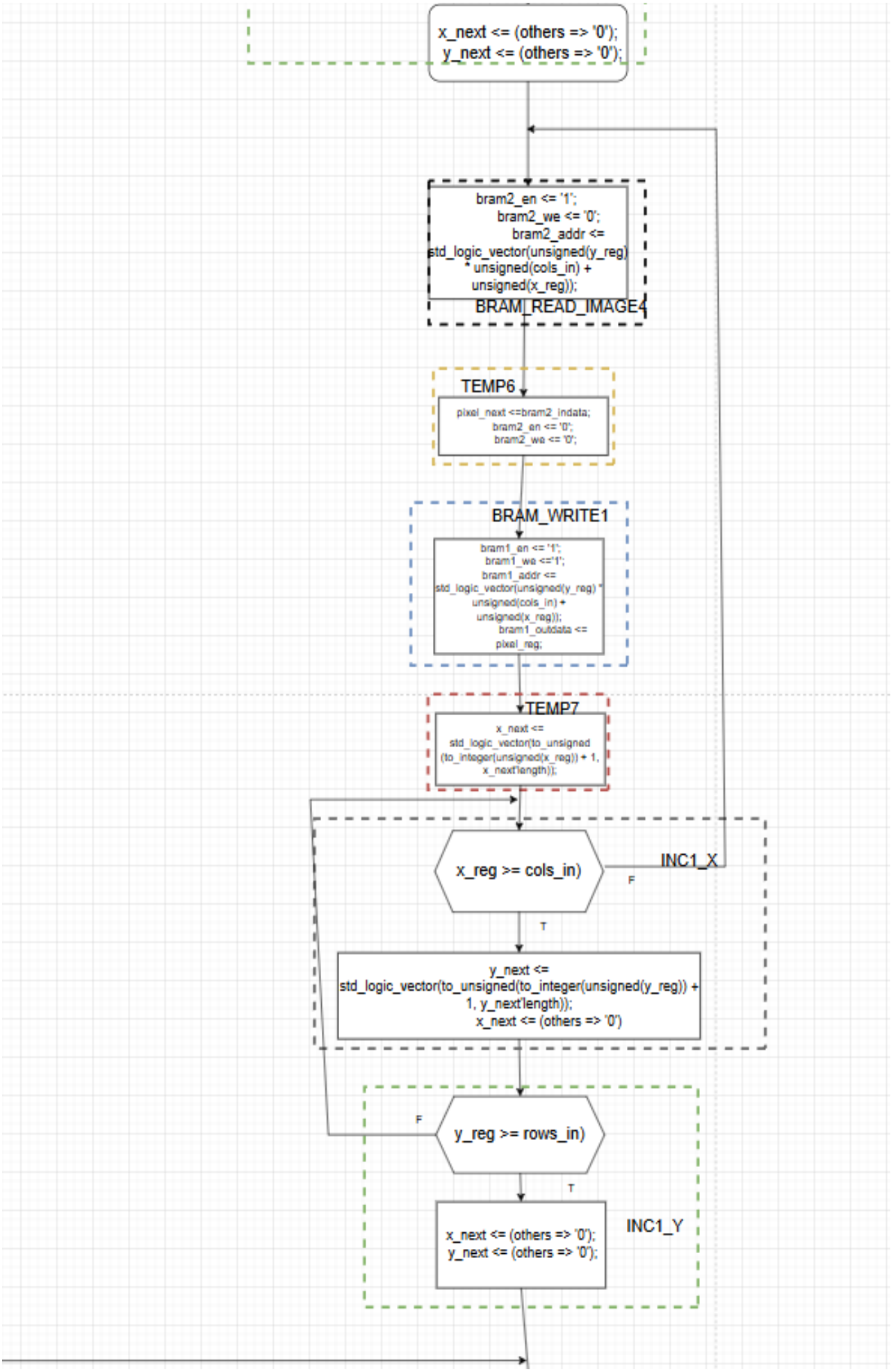
Ovaj dijagram predstavlja osnovu za sinhronizovano digitalno kolo koje može realizovati algoritam prepoznavanja boja i kontura direktno u hardveru, koristeći VHDL kao jezik za opis arhitekture.

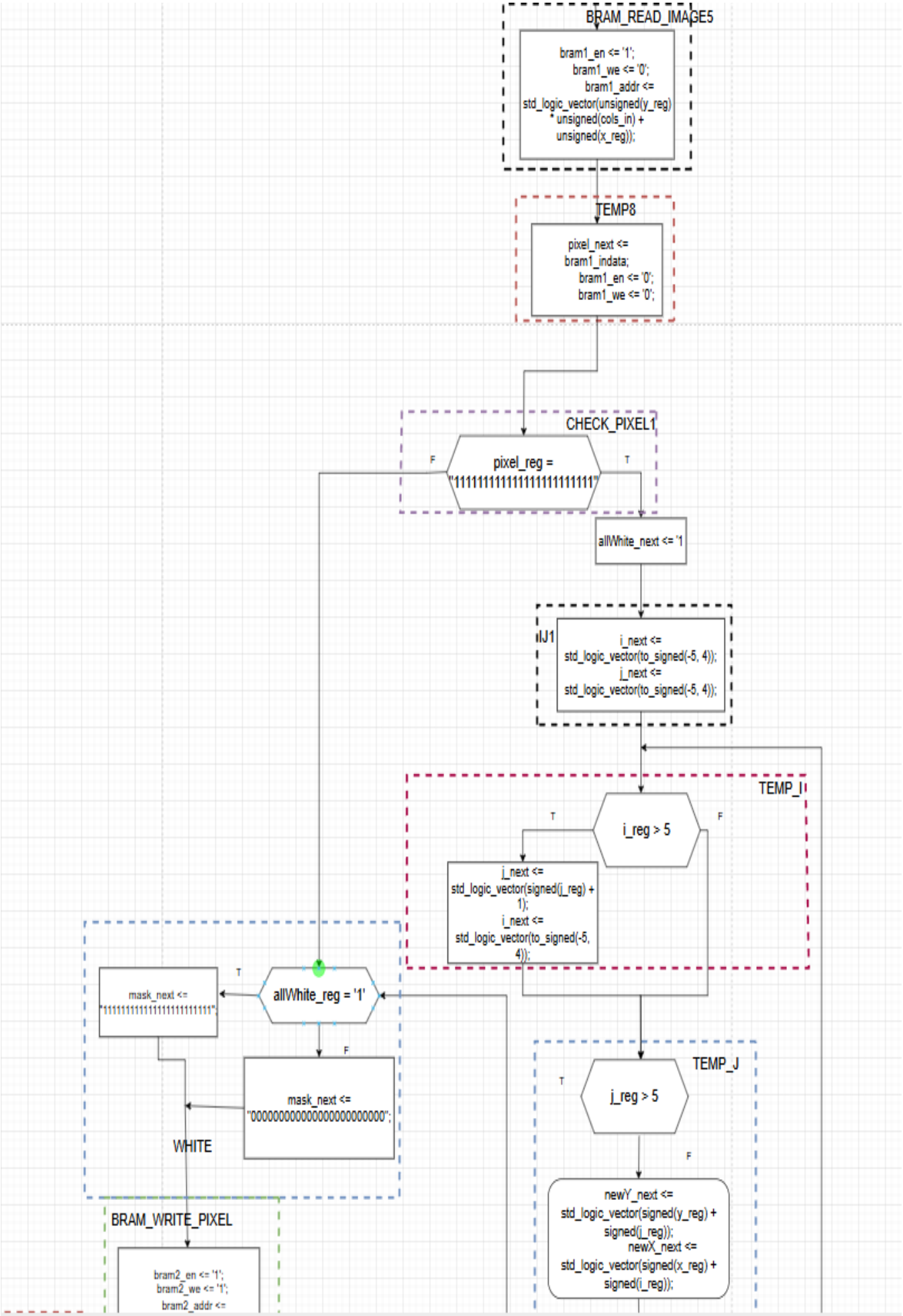


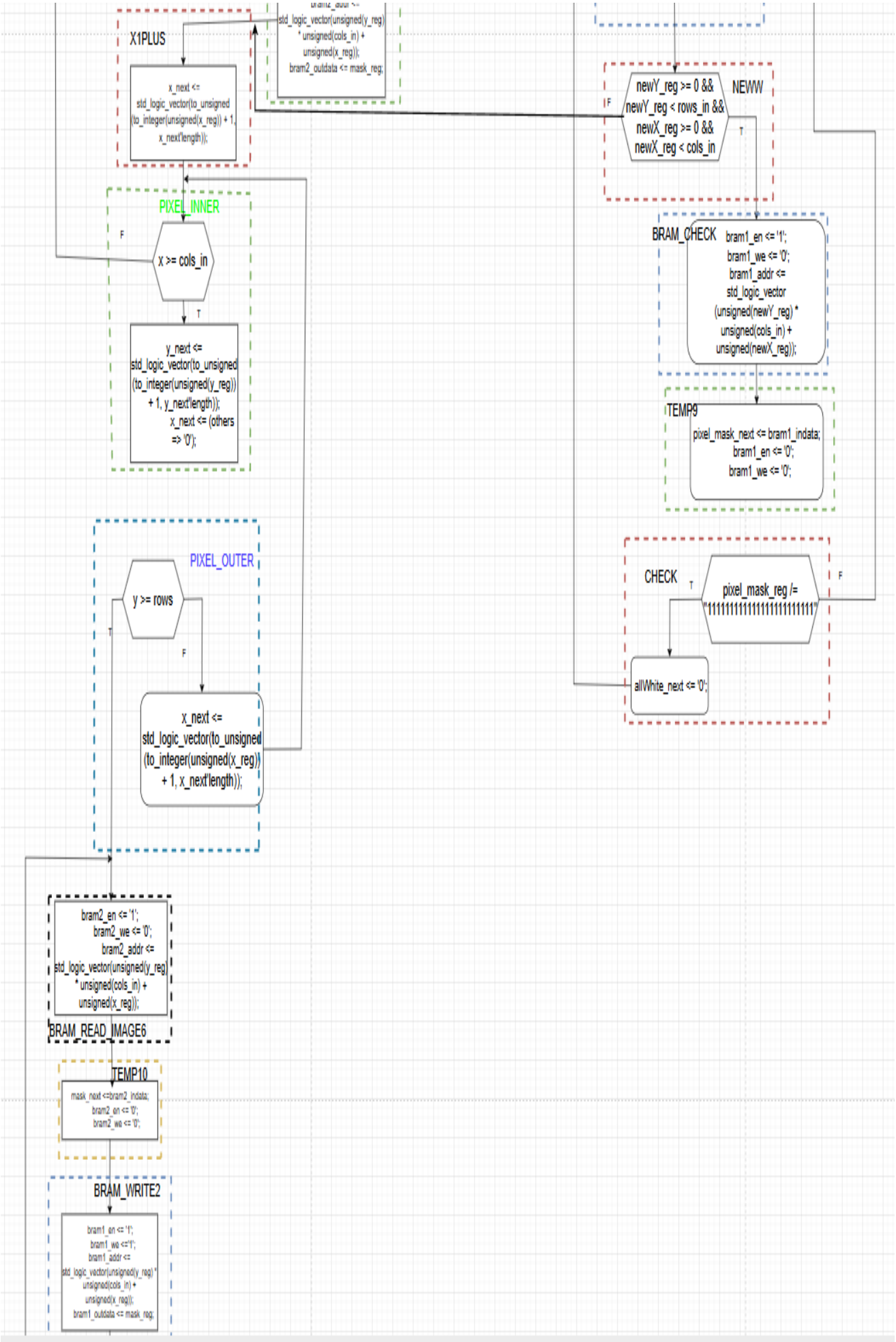

WRITE_TO_BRAM_MASK1
bram2_en <= '1';
bram2_we <= '1';
bram2_addr <=
std_logic_vector(unsigned(y_reg)
* unsigned(cols_in) +
unsigned(x_reg));
bram2_outdata <=
pixel_mask_reg;
TEMP
x_next <=
std_logic_vector(to_unsigned(to_integer(unsigned(x_reg))
+ 1, x_next'length));
MASK_INNER
x_reg >= cols_in)
T
F
y_next <=
std_logic_vector(to_unsigned(to_integer(unsigned(y_reg)) +
1, y_next'length));
x_next <= (others => '0')
MASK_OUTER
y_reg >= rows_in)
T
F
x_next <= (others => '0');
y_next <= (others => '0');
bram2_en <= '1';
bram2_we <= '0';
bram2_addr <=
std_logic_vector(unsigned(y_reg)
* unsigned(cols_in) +
unsigned(x_reg));
BRAM_READ_IMAGE2
TEMP2
pixel_mask_next <=bram2_indata;
bram2_en <= '0';
bram2_we <= '0';
BRAM_WRITE
bram1_en <= '1';
bram1_we <='1';
bram1_addr <=
std_logic_vector(unsigned(y_reg) *
unsigned(cols_in) +
unsigned(x_reg));
bram1_outdata <=
pixel_mask_reg;

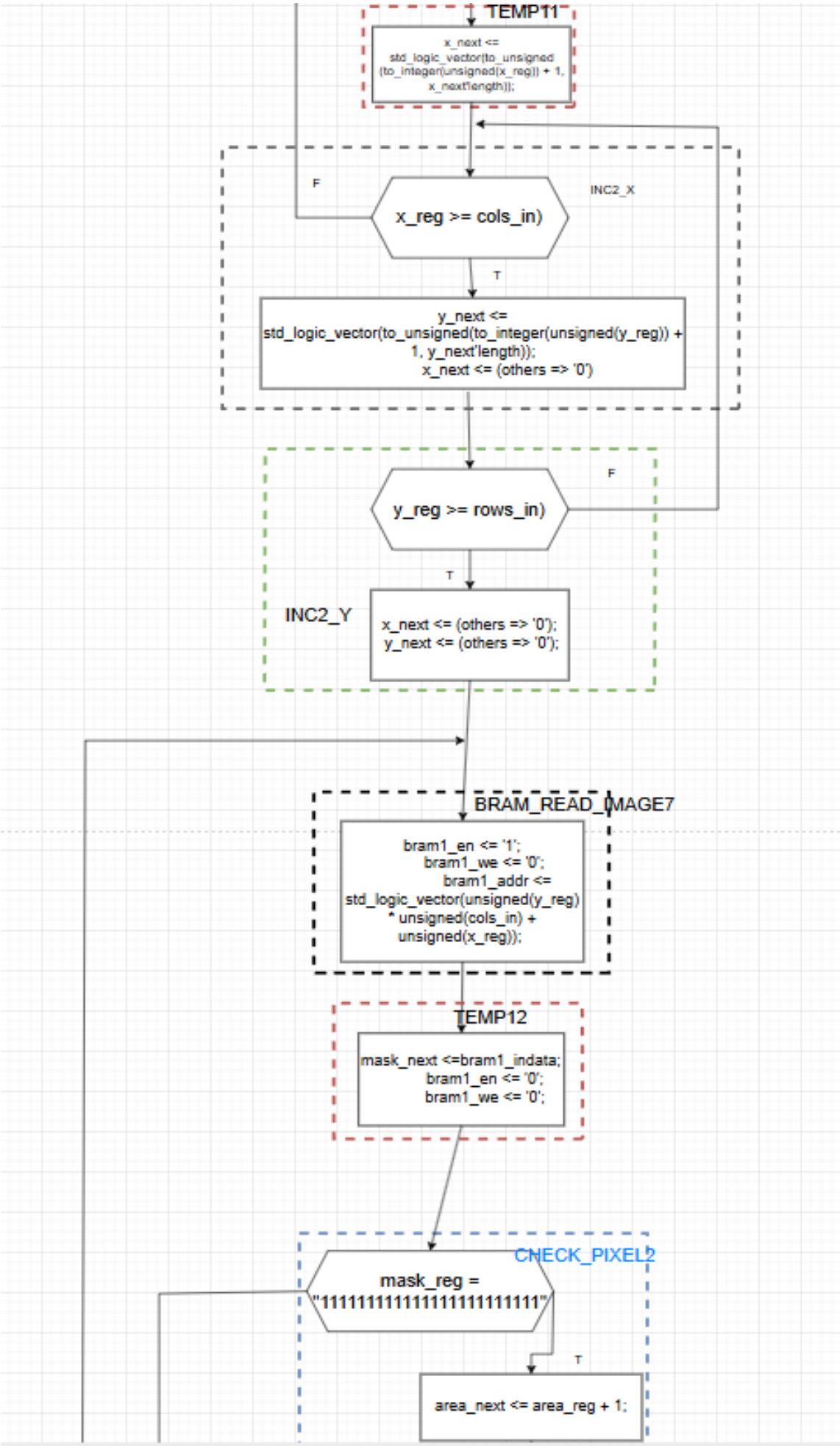

TEMP3
x_next <= std_logic_vector(to_unsigned (to_integer(unsigned(x_reg)) + 1, x_next'length));
x_reg >= cols_in)
F
INC_X
y_next <= std_logic_vector(to_unsigned(to_integer(unsigned(y_reg)) + 1, y_next'length));
x_next <= (others => '0')
y_reg >= rows_in)
F
INC_Y
T
x_next <= (others => '0');
y_next <= (others => '0');
bram1_en <= '1';
bram1_we <= '0';
bram1_addr <= std_logic_vector(unsigned(y_reg) * unsigned(cols_in) + unsigned(x_reg));
BRAM_READ_IMAGE3
TEMP4
pixel_mask_next <= bram1_indata;
bram1_en <= '0';
bram1_we <= '0';
F
pixel_mask_reg = "111111111111111111111111"
T
MASK
i_next <= std_logic_vector(to_signed(-5, 4));
j_next <= std_logic_vector(to_signed(-5, 4));
DILATION_I
T
i_reg > 5
F
j_next <= std_logic_vector(signed(j_reg) + 1);
i_next <= std_logic_vector(to_signed(-5, 4));


DILATION_J
T
j_reg > 5
F
newY_next <= std_logic_vector(signed(y_reg) + signed(j_reg));
newX_next <= std_logic_vector(signed(x_reg) + signed(i_reg));
DILATION_END
F
newY_reg >= 0 && newY_reg < rows_in && newX_reg >= 0 && newX_reg < cols_in
T
pixel_next <= "111111111111111111111111";
x_next <= std_logic_vector (to_unsigned(to_integer (unsigned(x_reg))+ 1, x_next'length));
XPLUS
WRITE_TO_BRAM_MASK2
bram2_en <= '1';
bram2_we <= '1';
bram2_addr <= std_logic_vector(unsigned(newY_reg) * unsigned(cols_in) + unsigned(newX_reg));
bram2_outdata <= pixel_reg;
TEMP5
_next <= std_logic_vector(signed(i_reg) + 1);
DILATION_INNER
F
x >= cols_in
T
y_next <= std_logic_vector(to_unsigned (to_integer(unsigned(y_reg)) + 1, y_next'length));
x_next <= (others => '0');
DILATION_OUTER
F
y >= rows_in

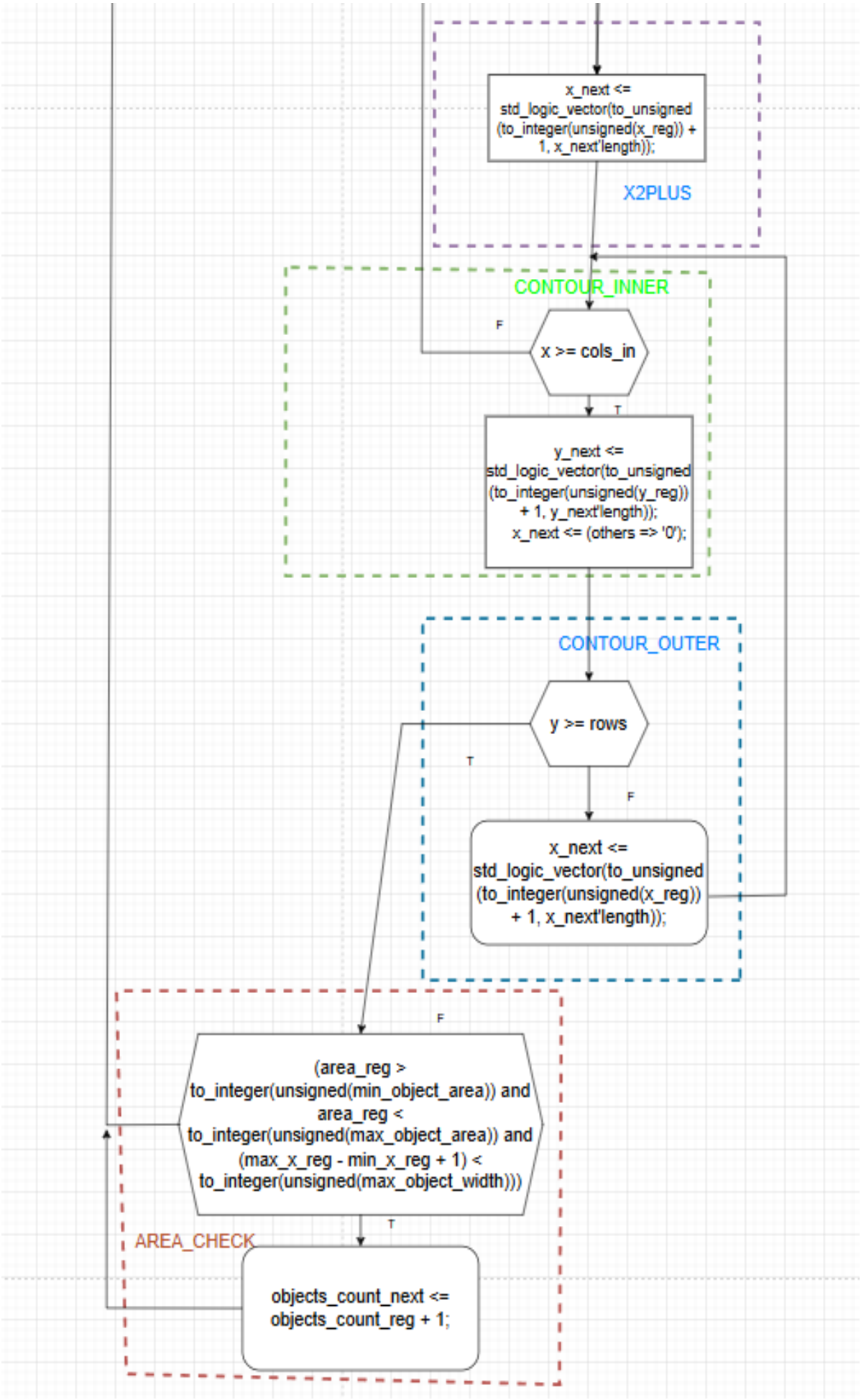

x_next <= (others => '0');
y_next <= (others => '0');
bram2_en <= '1';
bram2_we <= '0';
bram2_addr <=
std_logic_vector(unsigned(y_reg)
* unsigned(cols_in) +
unsigned(x_reg));
BRAM_READ_IMAGE4
TEMP6
pixel_next <=bram2_indata;
bram2_en <= '0';
bram2_we <= '0';
BRAM_WRITE1
bram1_en <= '1';
bram1_we <='1';
bram1_addr <=
std_logic_vector(unsigned(y_reg) *
unsigned(cols_in) +
unsigned(x_reg));
bram1_outdata <=
pixel_reg;
TEMP7
x_next <=
std_logic_vector(to_unsigned
(to_integer(unsigned(x_reg)) + 1,
x_next'length));
x_reg >= cols_in)
INC1_X
F
T
y_next <=
std_logic_vector(to_unsigned(to_integer(unsigned(y_reg)) +
1, y_next'length));
x_next <= (others => '0')
F
y_reg >= rows_in)
T
x_next <= (others => '0');
y_next <= (others => '0');
INC1_Y


BRAM_READ_IMAGE5
bram1_en <= '1';
bram1_we <= '0';
bram1_addr <=
std_logic_vector(unsigned(y_reg)
* unsigned(cols_in) +
unsigned(x_reg));
TEMP8
pixel_next <=
bram1_indata;
bram1_en <= '0';
bram1_we <= '0';
CHECK_PIXEL1
F
pixel_reg =
"111111111111111111111111"
T
allWhite_next <= '1
IJ1
i_next <=
std_logic_vector(to_signed(-5, 4));
j_next <=
std_logic_vector(to_signed(-5, 4));
TEMP_I
T
i_reg > 5
F
j_next <=
std_logic_vector(signed(j_reg) +
1);
i_next <=
std_logic_vector(to_signed(-5,
4));
T
mask_next <=
"111111111111111111111111";
allWhite_reg = '1'
F
mask_next <=
"000000000000000000000000";
WHITE
TEMP_J
T
j_reg > 5
F
newY_next <=
std_logic_vector(signed(y_reg) +
signed(j_reg));
newX_next <=
std_logic_vector(signed(x_reg) +
signed(i_reg));
BRAM_WRITE_PIXEL
bram2_en <= '1';
bram2_we <= '1';
bram2_addr <=


X1PLUS
x_next <= std_logic_vector(to_unsigned (to_integer(unsigned(x_reg)) + 1, x_next'length));
bram2_addr <= std_logic_vector(unsigned(y_reg) * unsigned(cols_in) + unsigned(x_reg)); bram2_outdata <= mask_reg;
newY_reg >= 0 && newY_reg < rows_in && newX_reg >= 0 && newX_reg < cols_in
NEWW
F
T
PIXEL_INNER
x >= cols_in
y_next <= std_logic_vector(to_unsigned (to_integer(unsigned(y_reg)) + 1, y_next'length)); x_next <= (others => '0');
BRAM_CHECK
bram1_en <= '1'; bram1_we <= '0'; bram1_addr <= std_logic_vector (unsigned(newY_reg) * unsigned(cols_in) + unsigned(newX_reg));
TEMP9
pixel_mask_next <= bram1_indata; bram1_en <= '0'; bram1_we <= '0';
PIXEL_OUTER
y >= rows
x_next <= std_logic_vector(to_unsigned (to_integer(unsigned(x_reg)) + 1, x_next'length));
CHECK
pixel_mask_reg /= "11111111111111111111111111"
allWhite_next <= '0';
bram2_en <= '1'; bram2_we <= '0'; bram2_addr <= std_logic_vector(unsigned(y_reg) * unsigned(cols_in) + unsigned(x_reg));
BRAM_READ_IMAGE6
TEMP10
mask_next <=bram2_indata; bram2_en <= '0'; bram2_we <= '0';
BRAM_WRITE2
bram1_en <= '1'; bram1_we <='1'; bram1_addr <= std_logic_vector(unsigned(y_reg) * unsigned(cols_in) + unsigned(x_reg)); bram1_outdata <= mask_reg;


TEMP11
x_next <=
std_logic_vector(to_unsigned
(to_integer(unsigned(x_reg)) + 1,
x_next'length));
F
INC2_X
x_reg >= cols_in)
T
y_next <=
std_logic_vector(to_unsigned(to_integer(unsigned(y_reg)) +
1, y_next'length));
x_next <= (others => '0')
F
y_reg >= rows_in)
T
INC2_Y
x_next <= (others => '0');
y_next <= (others => '0');
BRAM_READ_IMAGE7
bram1_en <= '1';
bram1_we <= '0';
bram1_addr <=
std_logic_vector(unsigned(y_reg)
* unsigned(cols_in) +
unsigned(x_reg));
TEMP12
mask_next <=bram1_indata;
bram1_en <= '0';
bram1_we <= '0';
CHECK_PIXEL2
mask_reg =
"111111111111111111111111"
T
area_next <= area_reg + 1;

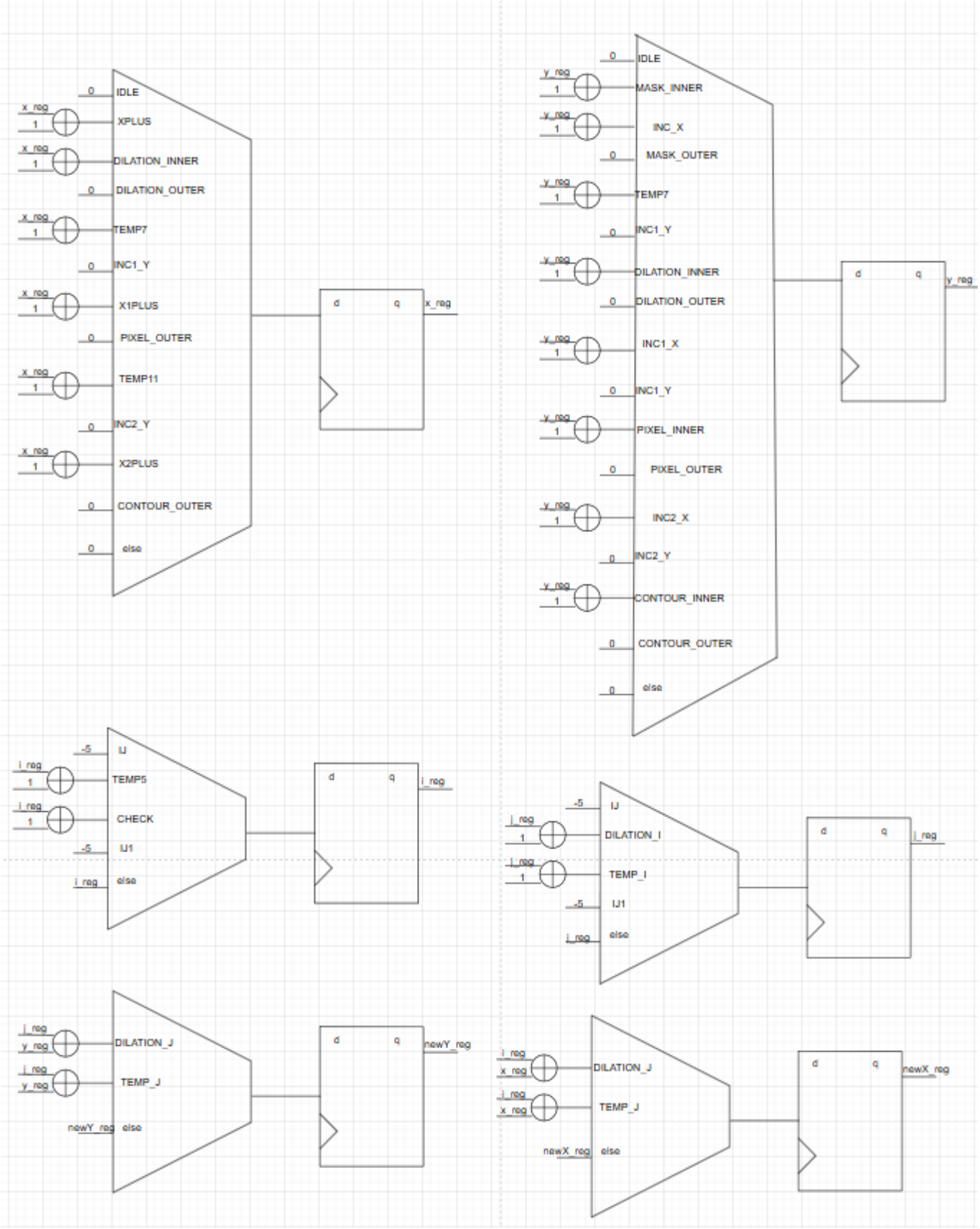
T
to_integer(unsigned(x_reg)) < min_x_reg
CHECK_MINX
F
min_x_next <= to_integer(unsigned(x_reg));
min_x_next <= min_x_reg;
T
to_integer(unsigned(x_reg)) > max_x_reg
F
CHECK_MAXX
max_x_next <= to_integer(unsigned(x_reg));
max_x_next <= max_x_reg;
T
to_integer(unsigned(y_reg)) < min_y_reg
F
CHECK_MINY
min_y_next <= to_integer(unsigned(y_reg));
min_y_next <= min_y_reg;
T
to_integer(unsigned(y_reg)) > max_y_reg
F
CHECK_MAXY
max_y_next <= to_integer(unsigned(y_reg));
max_y_next <= max_y_reg;

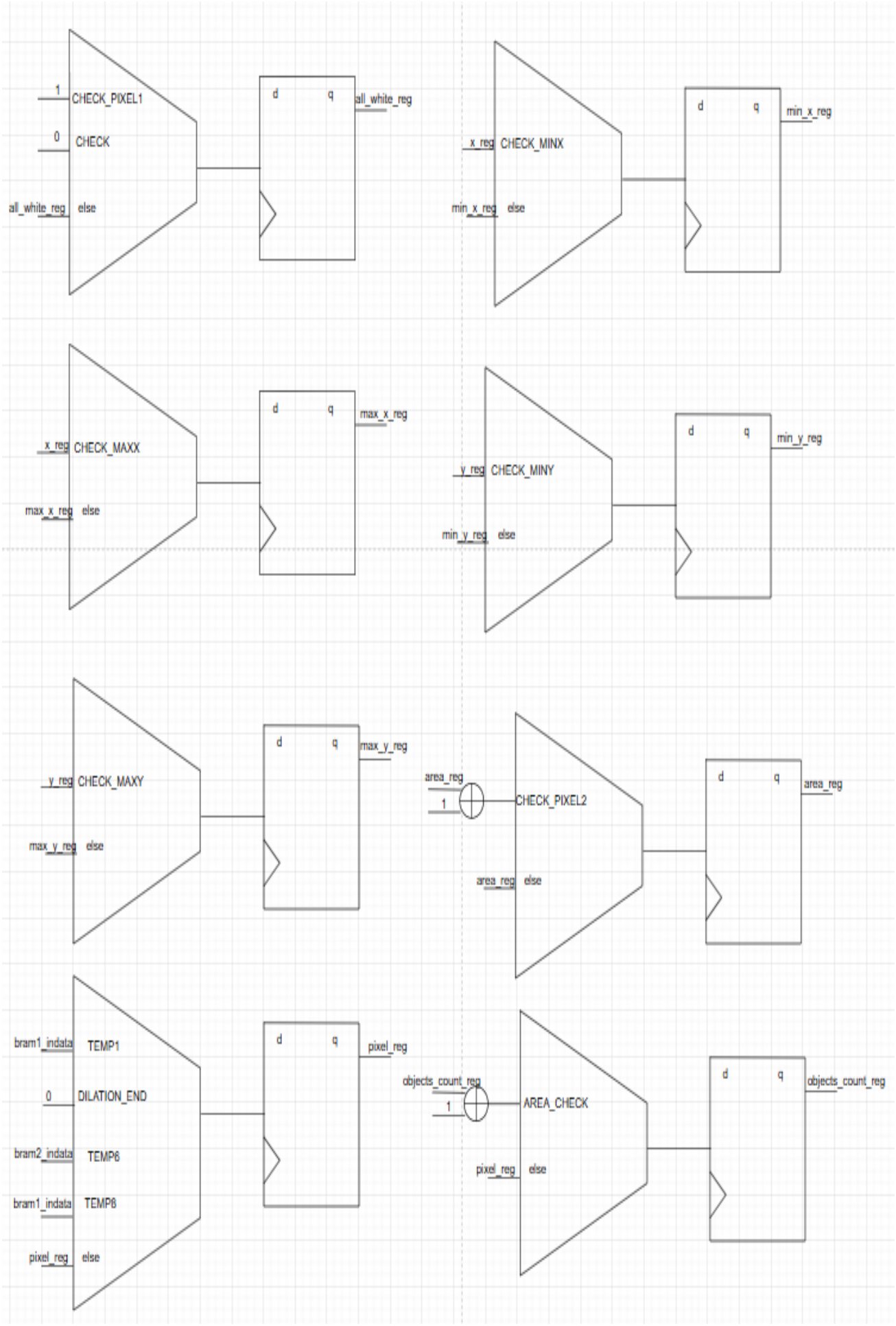


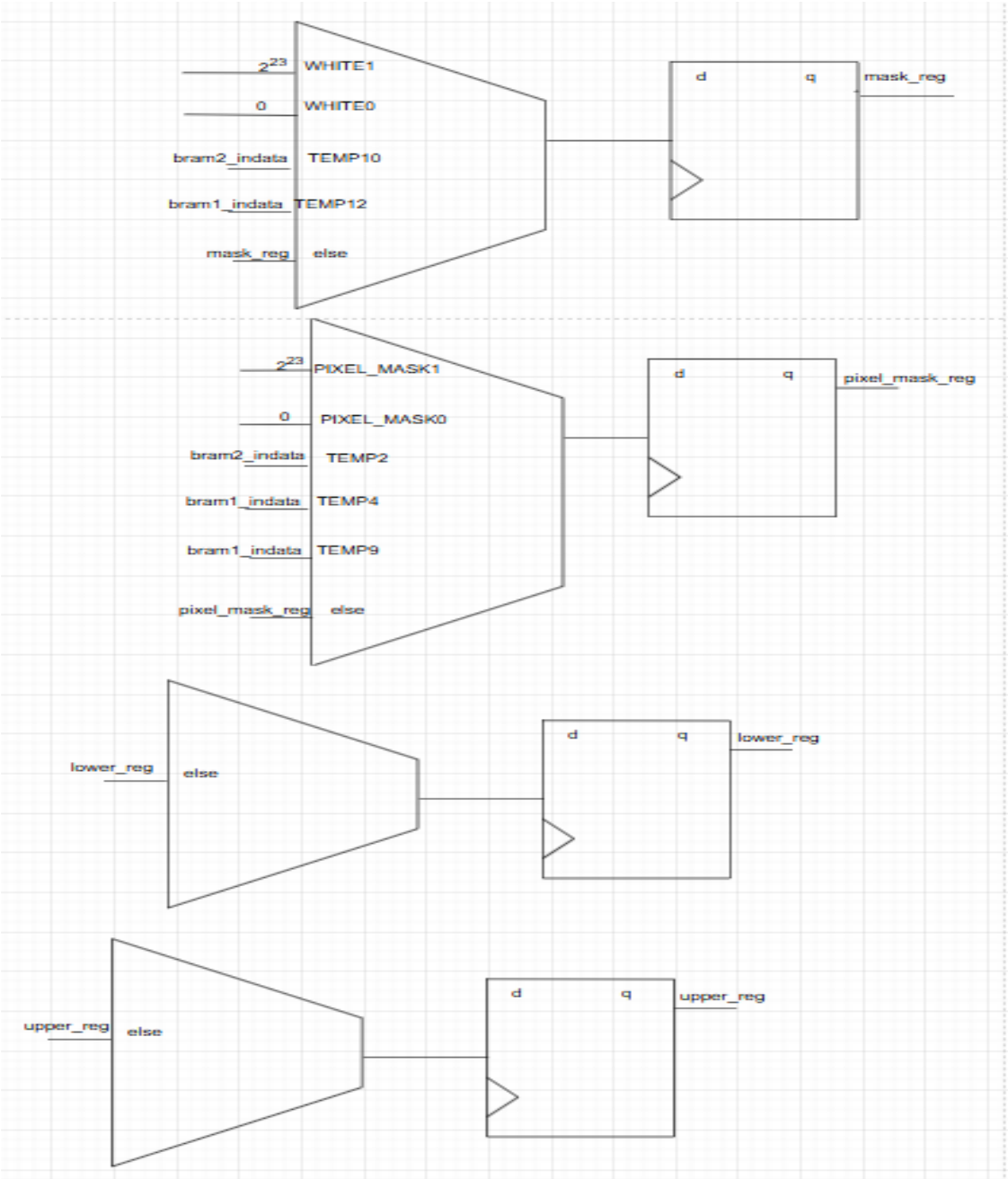
Slika 3, ASM Dijagram

4. Datapath sistema

Datapath sistema implementira sve aritmetičke, selekcione i memorijske operacije koje omogućavaju piksel-po-piksel obradu slike, uključujući maskiranje, morfološke operacije (dilataciju i eroziju), kao i detekciju kontura. Ovaj deo sistema je zadužen za prenos i manipulaciju podacima, dok FSM upravlja njihovim tokom.




1
CHECK_PIXEL1
0
CHECK
all_white_reg
else
d
q
all_white_reg
x_reg
CHECK_MINX
min_x_reg
else
d
q
min_x_reg
x_reg
CHECK_MAXX
max_x_reg
else
d
q
max_x_reg
y_reg
CHECK_MINY
min_y_reg
else
d
q
min_y_reg
y_reg
CHECK_MAXY
max_y_reg
else
d
q
max_y_reg
area_reg
1
CHECK_PIXEL2
area_reg
else
d
q
area_reg
bram1_indata
TEMP1
0
DILATION_END
bram2_indata
TEMP6
bram1_indata
TEMP8
pixel_reg
else
d
q
pixel_reg
objects_count_reg
1
AREA_CHECK
pixel_reg
else
d
q
objects_count_reg



Slika 4, Datapath sistema

5. Dizajn i interfejsi

5.1 IP jezgro

Ovo IP jezgro implementira algoritam segmentacije i obrade slike u hardveru koristeći finite state machine (FSM) arhitekturu. Modul radi na ulaznim podacima (piksela slike) i vrši sledeće čitanje slike iz interne BRAM memorije, thresholding piksela na osnovu zadatih donjih i gornjih granica boja (segmentacija po boji), kreiranje binarne maske na osnovu thresholding-a, proširenje bele oblasti u maski primenom morfološke operacije dilacije, izračunavanje karakteristika pronađenih objekata, kao što su minimalna i maksimaln površina i broj objekata, vrši upis rezultata u drugu BRAM memoriju za svaku narednu obradu. Prvo korisnik upisuje ulaznu binarnu sliku u bram1. Svaki piksel, prilikom niza operacija nad slikom, se upisuje na adresu y * cols_in + x. IP jezgro nakon start signala izvodi sledeće:

- Čita podatke iz bram1,
- Radi morfološke operacije, proveru maski i logiku validacije objekata,
- Srednju obradu i modifikovanu masku upisuje u bram2,

- Kopira rezultat iz bram2 nazad u bram1,
- Detektuje objekte i meri njihove karakteristike,
- Na osnovu postavljenih kriterijuma odlučuje koji objekti se priznaju.

Nakon završetka obrade korisnik dobija sledeće finalna rezultat slike u vidu binarnih vrednosti, kao i broj detektovanih objekata na izlaznom portu objects_count_out, ako objekti na slici zadovoljavaju sve uslove vezano za opseg boja, širinu, visinu i površinu.

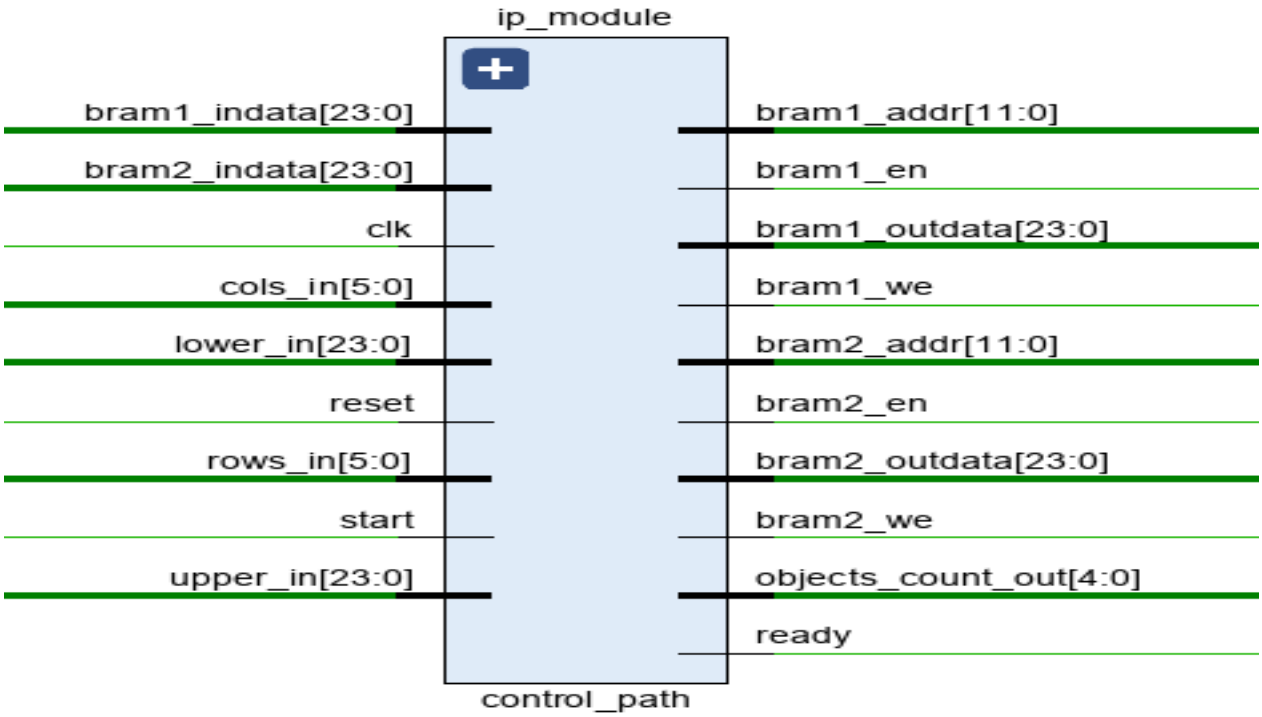
5.1.1 Arhitektura modula

Sekvencijalni deo (proces sinhronizovan na takt) realizuje skladištenje i ažuriranje stanja FSM i pomoćnih registara. Na svaki takt (clk) ažurira interne registre kao što su pozicije piksela (x_reg, y_reg), brojači, stanje FSM (current_state), i druge kontrolne signale. Sekvencijalni deo Implementira resetovanje modula na početne vrednosti. Takođe, ovaj deo je zadužen za "pamćenje" stanja između taktova i omogućava sinhronizovanu kontrolu toka podataka.

Kombinacioni deo izračunava sledeće vrednosti registara i sledeće stanje FSM na osnovu trenutnog stanja i ulaznih signala, sadrži logiku odlučivanja o prelazima između stanja FSM,realizuje uslove za thresholding i generisanje binarne maske, implementira algoritam morfološke dilatacije kroz analizu okolnih piksela maske, izračunava i ažurira parametre objekata (min/max koordinate, površinu), kontroliše čitanje i pisanje u BRAM memorije.

5.1.2 Tok rada modula

Modul čeka signal start. Kada se pokrene, čita piksele slike iz BRAM memorije red po red. Za svaki piksel proverava da li spada u zadati opseg boja (lower i upper). Kreira binarnu masku, gde su pikseli u opsegu označeni kao beli, ostali kao crni. Kada se napravi maska, primjenjuje se morfološka dilatacija na masku radi uklanjanja šuma i povezanosti objekata. Nakon toga računaju se geometrijske karakteristike pronađenih objekata (površinu i granice), a validni objekti koji se računaju kao pronađeni moraju da ispunjavaju pomenute karakteristike. Na kraju, signal ready prelazi u stanje logičke nule i označava kraj obrade.



Slika 5, IP modul

broj	PORT NAME	IN/OUT	WIDTH
1	clk	in	std_logic
2	reset	in	std_logic
3	start	in	std_logic
4	ready	out	std_logic
5	rows_in	in	std_logic_vector(5 downto 0)
6	cols_in	in	std_logic_vector(5 downto 0)
7	lower_in	in	std_logic_vector(23 downto 0)
8	upper_in	in	std_logic_vector(23 downto 0)
9	bram1_en	out	std_logic
10	bram1_we	out	std_logic
11	bram1_addr	out	std_logic_vector(11 downto 0)
12	bram1_indata	in	std_logic_vector(23 downto 0)
13	bram1_outdata	out	std_logic_vector(23 downto 0)
14	bram2_en	out	std_logic
15	bram2_we	out	std_logic
16	bram2_addr	out	std_logic_vector(11 downto 0)
17	bram2_indata	in	std_logic_vector(23 downto 0)
18	bram2_outdata	out	std_logic_vector(23 downto 0)
19	objects_count_out	out	std_logic_vector(4 downto 0)

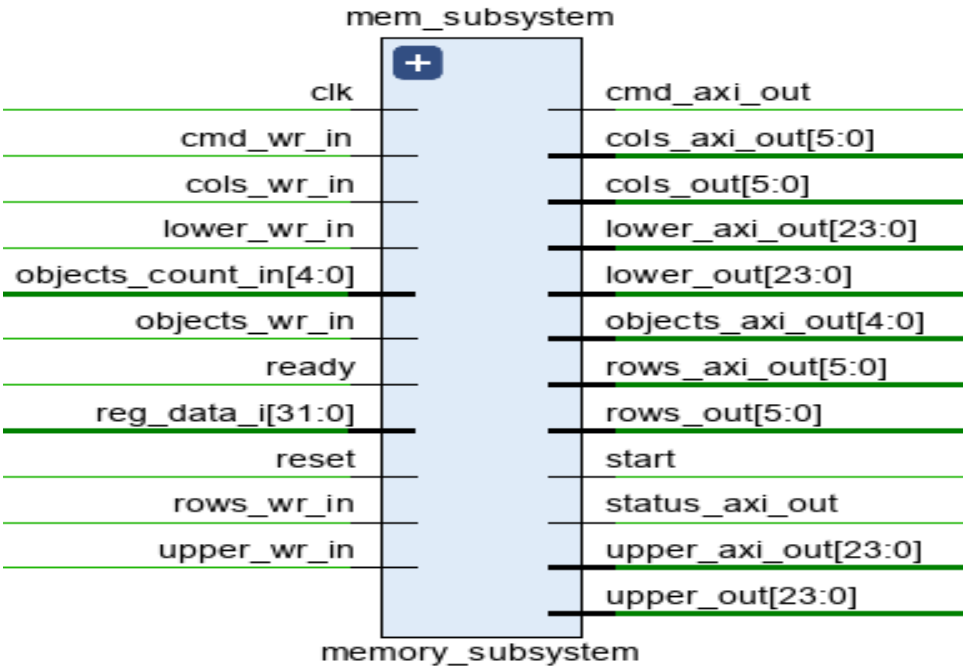
Tabela 1, Interfejs IP-a

5.2 Memory subsystem

Ovaj modul je namenski kontroler i skladište za upravljanje konfiguracionim parametrima i statusnim signalima između AXI magistrale (procesora) i glavnog IP jezgra koje obrađuje sliku. Memory subsystem upravlja:

- Konfiguracionim parametrima slike kao što su broj redova (rows), broj kolona (cols), donja (lower) i gornja (upper) granica boja za segmentaciju slike.
- Primanjem i prosleđivanjem broja detektovanih objekata dobijenih iz IP jezgra (broj objekata koje je sistem prepoznao).
- Interfejsom prema AXI magistrali što omogućava čitanje i pisanje pomenutih registara (parametara i statusa) putem AXI signala.
- Sinhronizacijom start i ready signala između IP jezgra i kontrolera, signal za start obrade (start) i signal koji pokazuje da je IP jezgro spremno (ready).
- Čuvanjem i ažuriranjem svih ovih registara u internim signalima (rows_sig, cols_sig, lower_sig, upper_sig, objects_count_sig, command_sig, status_sig) koji su dostupni i IP jezgru i AXI strani.

Modul prima vrednosti preko AXI interfejsa (u reg_data_i) i na osnovu kontrolnih signala (rows_wr_in, cols_wr_in, lower_wr_in, upper_wr_in, objects_wr_in, cmd_wr_in) upisuje te vrednosti u interne registre. Iz internih registara izbacuje izlaze prema IP jezgru (rows_out, cols_out, lower_out, upper_out, start) i prema AXI magistrali (rows_axi_out, cols_axi_out, itd.). Istovremeno prati status preko signala ready sa IP jezgra i ažurira status izlaz status_axi_out i omogućava pokretanje IP jezgra komandnim signalom (command_sig).



Slika 6, Memory subsystem

broj	PORT NAME	IN/OUT	WIDTH
1	clk	in	std_logic
2	reset	in	std_logic
3	start	out	std_logic
4	ready	in	std_logic
5	rows_out	out	std_logic_vector(5 downto 0)
6	cols_out	out	std_logic_vector(5 downto 0)
7	lower_out	out	std_logic_vector(23 downto 0)
8	upper_out	out	std_logic_vector(23 downto 0)
9	objects_count_in	in	std_logic_vector(4 downto 0)
10	reg_data_i	in	std_logic_vector(31 downto 0)
11	rows_wr_in	in	std_logic
12	cols_wr_in	in	std_logic
13	lower_wr_in	in	std_logic
14	upper_wr_in	in	std_logic
15	objects_wr_in	in	std_logic
16	rows_axi_out	out	std_logic_vector(5 downto 0)
17	cols_axi_out	out	std_logic_vector(5 downto 0)
18	lower_axi_out	out	std_logic_vector(23 downto 0)
19	upper_axi_out	out	std_logic_vector(23 downto 0)
20	objects_axi_out	out	std_logic_vector(4 downto 0)
21	cmd_wr_in	in	std_logic
22	cmd_axi_out	out	std_logic
23	status_axi_out	out	std_logic

Tabela 2, Interfejs memory subsystem

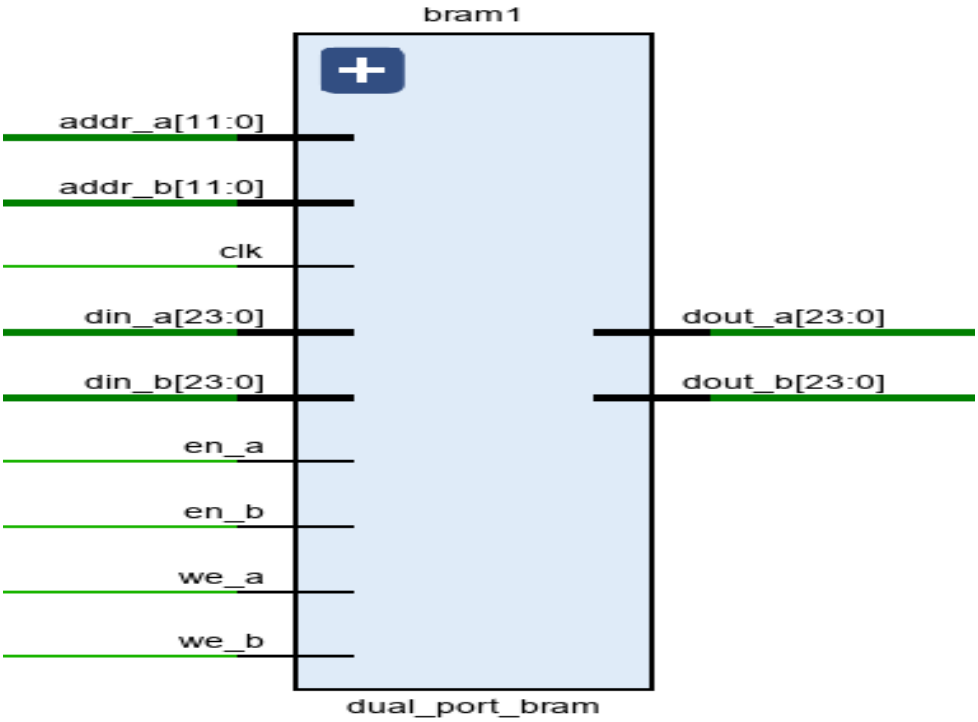
5.3 Dual port bram

Modul dual_port_bram predstavlja dvostrano-portnu sinhronu memoriju širine 24 bita i dubine 3721 lokacija. Omogućava istovremeni čitanje i/ili pisanje sa dva nezavisna porta (A i B), čime omogućava visoku

propusnost pri obradi slike. Služi kao međumemorija za skladištenje piksela slike i komunikaciju između kontrolne logike i računskog jezgra.

U okviru IP jezgra koriste se dve dual-port BRAM memorije za skladištenje slike i međurezultata obrade. BRAM1 se koristi kao primarna memorija slike. Na početku rada, slika se učitava iz procesora direktno u BRAM1 putem AXI interfejsa i odgovarajuće kontrolne logike. Nakon učitavanja, IP jezgro započinje obradu slike. Obrađeni rezultati svakog koraka (npr. filtriranja, segmentacije i sl.) upisuju se u BRAM2. Kada se jedan korak obrade završi, sadržaj BRAM2 (obrađena slika) se kopira nazad u BRAM1, čime BRAM1 uvek sadrži trenutno najnoviju verziju slike. Sledeći korak obrade se zatim izvršava koristeći ažuriranu verziju slike iz BRAM1, a novi rezultati se ponovo upisuju u BRAM2.

Ovakav mehanizam omogućava naizmenično korišćenje BRAM1 i BRAM2 za čuvanje ulaznih i izlaznih podataka u svakom ciklusu obrade, čime se izbegava potreba za dodatnim privremenim baferima i obezbeđuje efikasna i organizovana obrada slike.

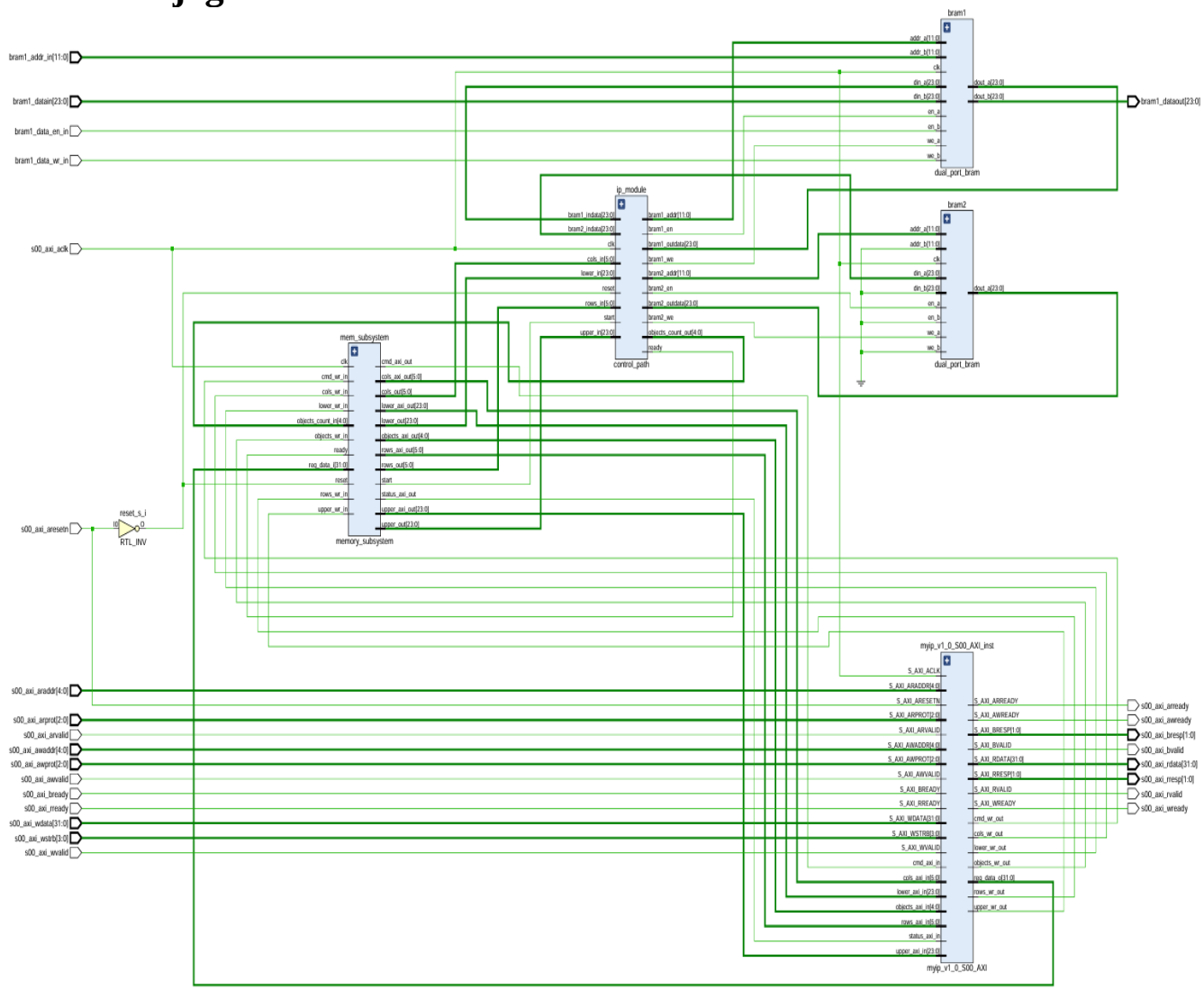


Slika 7, Bram

broj	GENERIC NAME	TYPE	VALUE
1	DATA_WIDTH	integer	24
2	ADDR_WIDTH	integer	12
broj	PORT NAME	IN/OUT	WIDTH
1	clk	in	std_logic
e	en_a	in	std_logic
3	en_b	in	std_logic
4	we_a	in	std_logic
5	we_b	in	std_logic
6	addr_a	in	std_logic_vector(ADDR_WIDTH-1 downto 0)
7	addr_b	in	std_logic_vector(ADDR_WIDTH-1 downto 0)
8	din_a	in	std_logic_vector(DATA_WIDTH-1 downto 0)
9	din_b	in	std_logic_vector(DATA_WIDTH-1 downto 0)
10	dout_a	out	std_logic_vector(DATA_WIDTH-1 downto 0)
11	dout_b	out	std_logic_vector(DATA_WIDTH-1 downto 0)

Tabela 3, Interfejs Bram-a

5.4 Blok dijagram sistema



Slika 8, Blok dijagram sistema

6. Analiza modelovanog sistem nakon sinteze i implementacije

Nakon završene sinteze i implementacije modelovanog sistema, izvršena je detaljna analiza njegovih performansi, zauzeća resursa i funkcionalnosti, čiji će rezultati biti prikazani u nastavku.

6.1 Analiza utrošenih resursa

Nakon sinteze sistema u Vivado alatu, analizirani su utrošeni resursi, uključujući broj korišćenih logičkih ćelija (LUT), flip-flopova (FF), blok RAM memorija i ostalih resursa. U konkretnom slučaju prikazani su iskorištenosti LUT-ova, FF-ova i DSP-ova.

Resource	Utilization	Available	Utilization %
LUT	487	17600	2.77
FF	185	35200	0.53
DSP	12	80	15.00

Slika 9, Analiza utrošenih resursa

6.2 Analiza kritičnih putanja i maksimalne frekvencije rada sistema

Za analizu tajminga sistema, korišten je takt sa periodom 8.5 ns što odgovara frekvenciji od 117.6 MHz i na osnovu rezultata statičke vremenske analize, dobijeni su sledeći parametri:

- Worst Negative Slack (WNS): 0.392 ns
- Worst Hold Slack (WHS): 0.224 ns
- Worst Pusle Width Slack (WPWS)č 0.6 ns

Pozitivan WNS pokazuje da najkritičnija putanja ima višak vremena od 0.392 ns u odnosu na zadati period. To ukazuje da sistem može potencijalno da radi i na većoj frekvenciji.

Na osnovu toga, teoretska maksimalna frekvencija sistema se procenjuje na:

$$\boldsymbol{T_{krit} = 8.5\ ns - 0.392\ ns = 8.108\ ns \Rightarrow f_{max} \approx 1/8.108\ ns \approx 123.4\ MHz}$$

Iako sistem radi stabilno na 117.6 MHz, u teoriji može da dostigne i do 123.4 MHz ukoliko se dizajn dodatno optimizuje.

6.3 Određivanje propusne moći (throughput) i kašnjenja (latency) sistema

Analizom implementiranog digitalnog sistema njegova struktura je sekvencijalna, zasnovana na automatu sa konačnim brojem stanja (FSM), bez upotrebe pipelajna. Svaki piksel slike se obrađuje kroz niz uzastopnih stanja, pri čemu se obrada narednog piksela započinje tek nakon završetka prethodnog. Na osnovu vremenskih karakteristika period takta je 8.5 ns, a frekvencija takta je 117.6 MHz.

Kašnjenje (latency) - Obrada jednog piksela zahteva prelazak kroz 39 sekvencijalnih stanja (čitanje piksela iz memorije, privremeno registrovanje, logička provera, upis maske i priprema za sledeći piksel). Ukupno trajanje obrade jednog piksela iznosi:

Latency = (39+13) * 8.5 ns = 442 ns

Propusna moć (throughput) - S obzirom da sistem nije pipelajnovan i da se svaki piksel obrađuje kroz 39 taktova plus 13 ciklusa što se utroši na upis ili čitanje BRAM-a, maksimalna propusna moć sistema za slike 400x200 iznosi:

Throughput = 1/(80000piksela*52takta*8.5ns) = 28 slika u sekundi

6.4 Snaga

Na osnovu Power Analysis izveštaja nakon implementacije, ukupna potrošnja sistema je sledeća:

- Ukupna snaga na čipu (Total On-Chip Power): 0.103 W
- Dinamička snaga (Dynamic Power): 0.013 W (13%)

Od toga:

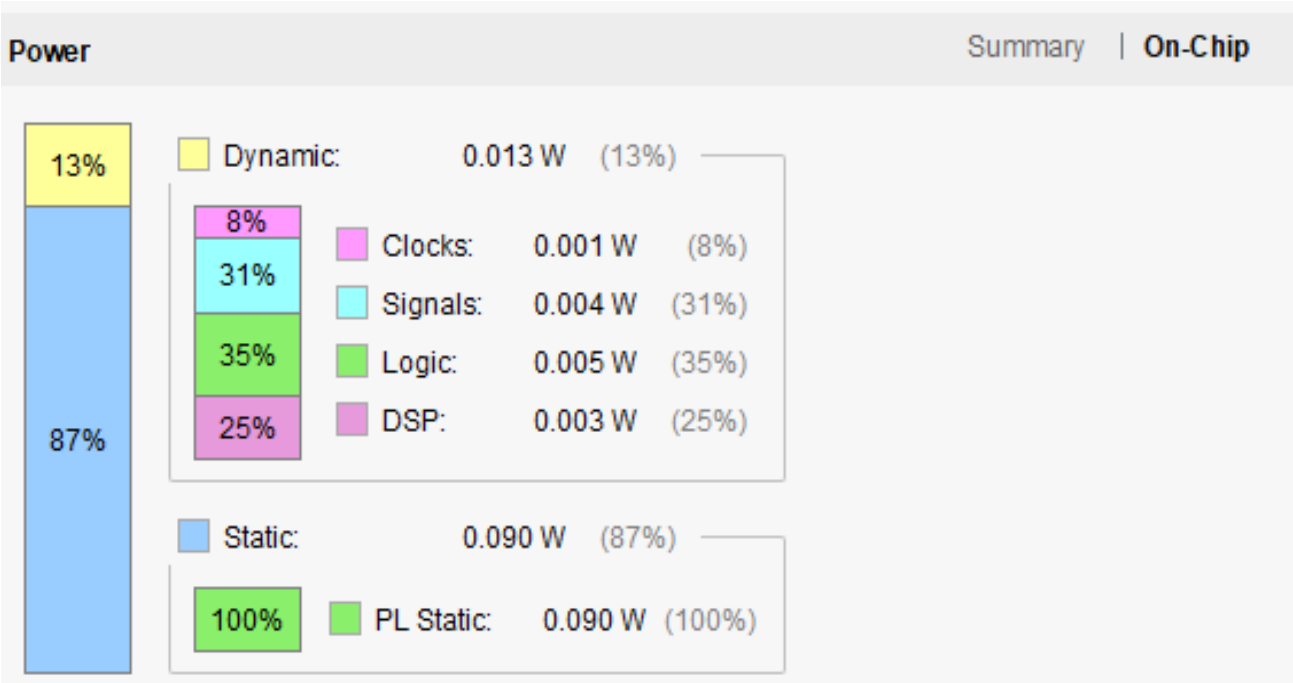
- Clock mreža: 0.001 W (8%)
- Signali: < 0.004 W (31%)
- Logika: < 0.005 W (35%)
- Statika uređaja (Device Static): 0.003 W (25%)

Dodatni parametri:

- Temperatura spoja (Junction Temperature): 26.2°C

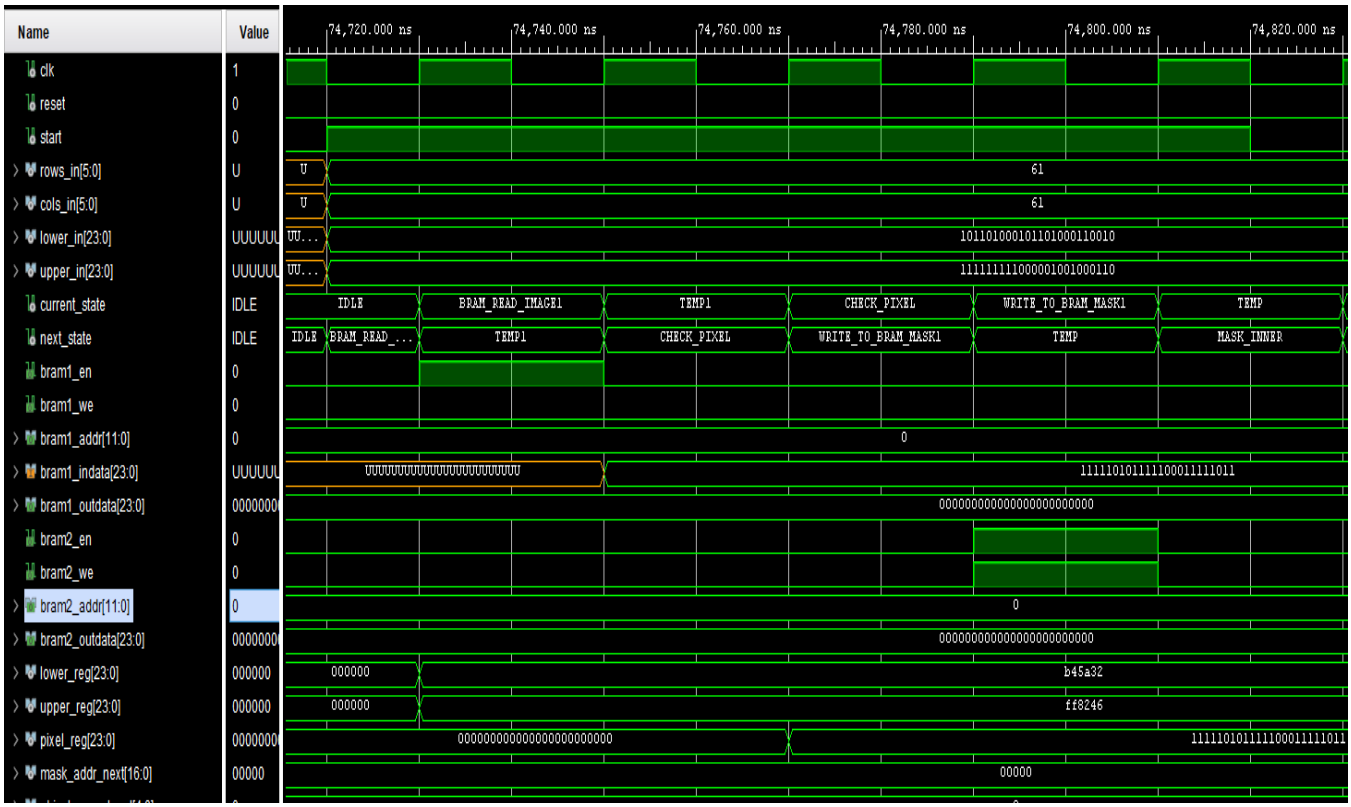
- Termalna margina: 58.8°C
- Efektivni otpor toplotnog prelaza (Effective ΘJA): 11.5°C/W
- Snaga isporučena uređajima van čipa: 0 W

Na osnovu ovih rezultata, može se zaključiti da dizajn ima nisku potrošnju energije, a najveći deo ukupne snage otpada na statičku komponentu, dok je dinamička snaga veoma niska.

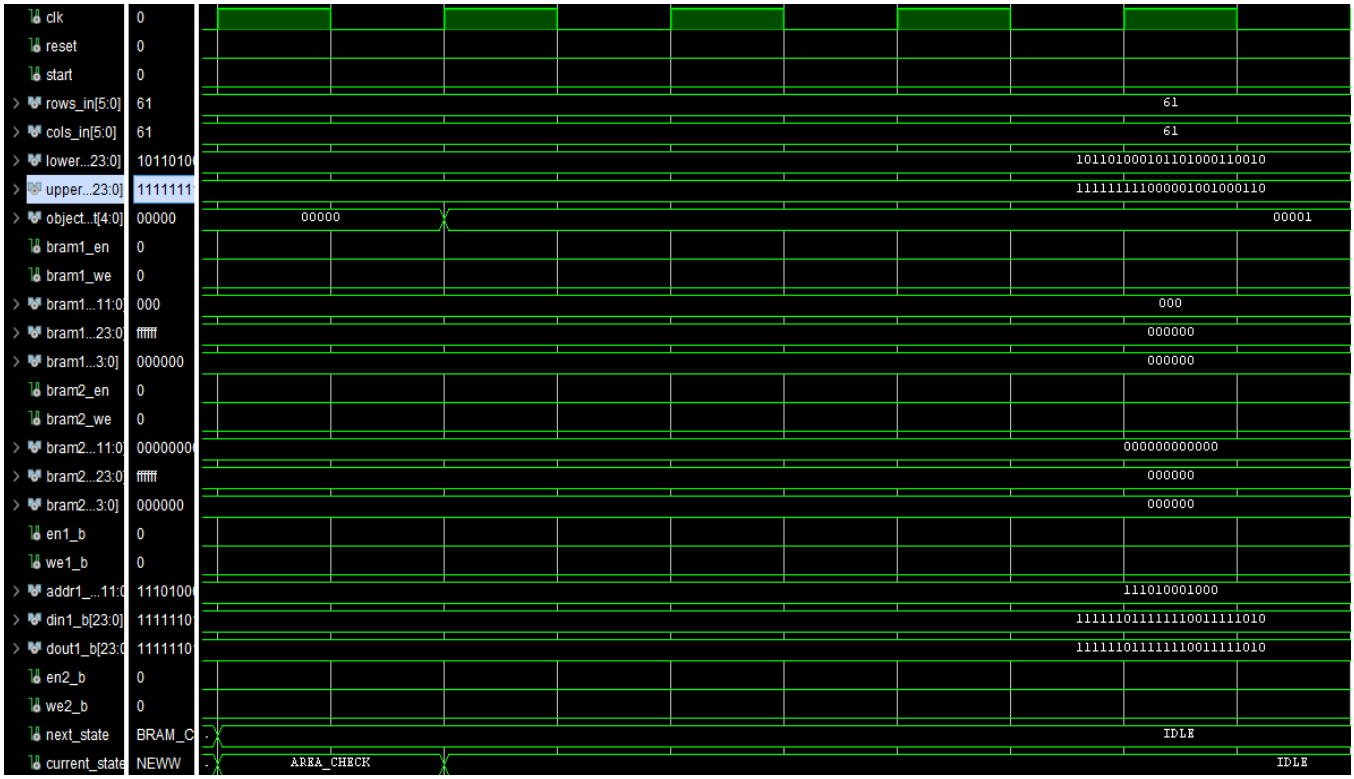


Slika 10, Snaga

6.5 Rezultati simulacije



Slika 11, Prvi deo simulacije

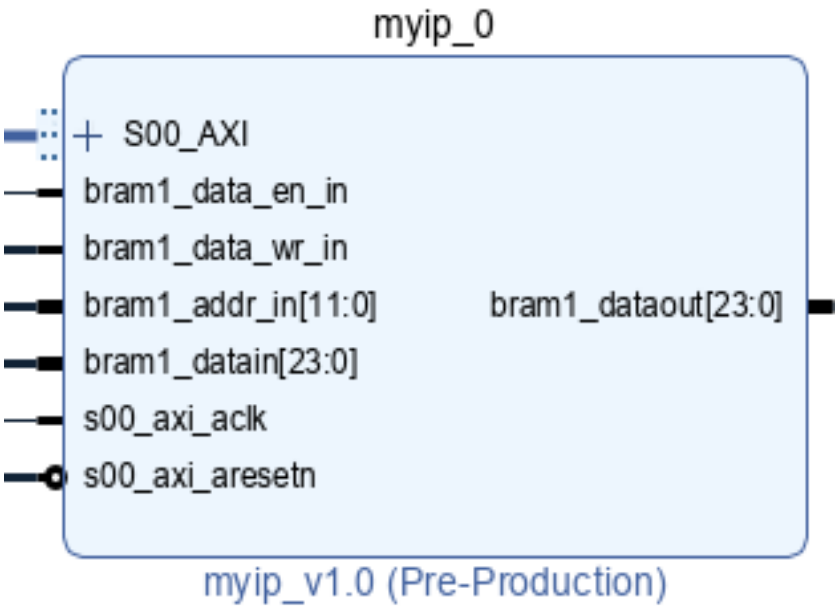


Slika 12, Drugi deo simulacije

7. Pakovanje sistema u IP jezgro

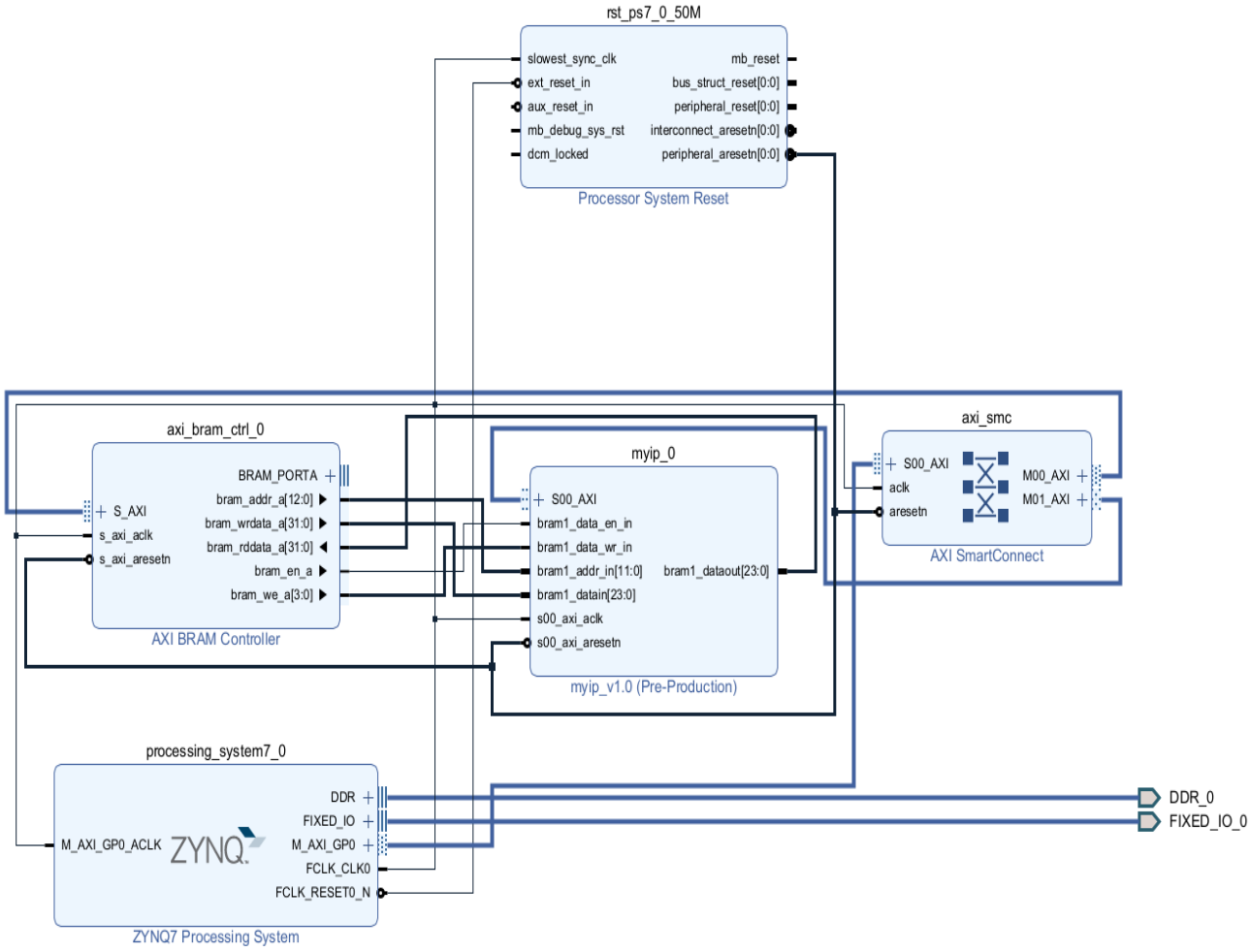
Sistem je upakovan u IP (Intellectual Property) jezgro radi modularnosti i lakše integracije u veće hardverske sisteme. Pakovanje u IP jezgro omogućava da se dizajn koristi kao samostalna, ponovo upotrebljiva komponenta koja može lako da se poveže sa drugim IP-jevima ili procesorima kroz standardizovane interfejse (AXI4-Lite).

Razlozi pakovanja u IP jezgro je da bi se omogućila modularnost i ponovna upotreba, jednostavna integracija, standardizovani interfejs, lakša verifikacija i održavanje i omogućava podešavanje parametara jezgra prilikom integracije, čime se povećava fleksibilnost korišćenja.



Slika 13, Zapakovano IPjezgro

8. Blok dizajn integrisanog sistema



Slika 14, Sistem povezan u Zync integratoru

9. Analiza integrisanog sistema nakon sinteze i implementacije i simulacije

Nakon integracije IP jezgra u širi sistem, izvršena je detaljna analiza funkcionisanja integrisanog sistema kroz faze sinteze i implementacije čiji će rezultati biti prikazani u nastavku.

9.1 Analiza utrošenih resursa

Nakon sinteze sistema u Vivado alatu, analizirani su utrošeni resursi, uključujući broj korišćenih LUT-ova, LUTRAM-ova, FF-ova, blok RAM memorija, DSP-a i BUFG.

Resource	Utilization	Available	Utilization %
LUT	4367	17600	24.81
LUTRAM	615	6000	10.25
FF	4543	35200	12.91
BRAM	3	60	5.00
DSP	12	80	15.00
BUFG	1	32	3.13

Slika 15, Rezultati sinteze

Ova raspodela pokazuje da je dizajn efikasno implementiran sa umerenim korišćenjem svih ključnih resursa. Najveće zauzeće je na LUT-ovima, što ukazuje na intenzivnu logičku obradu, ali i dalje unutar granica koje omogućavaju značajnu mogućnost proširenja. Korišćenje BRAM i DSP jedinica je takođe umereno, što govori da obrada podataka uključuje i rad sa memorijom kao i aritmetičke operacije. Samo 1 BUFG je iskorišćen, što znači da dizajn koristi samo jedan glavni takt, što pojednostavljuje sinhronizaciju i dizajn.

Ukupna iskorišćenost resursa pokazuje da je dizajn hardverski optimalan, stabilan i ima prostor za dodavanje novih funkcionalnosti ili optimizaciju performansi.

9.2 Analiza kritičnih putanja i maksimalne frekvencije rada sistema

Analizom vremenskih karakteristika digitalnog sistema dobijeni su sledeći rezultati:

- Period takta: 20 ns
- Frekvencija takta:

$$\mathbf{f = 1/T = 1/(20\times10^{-9}) \approx 50MHz}$$

  - Worst Negative Slack (WNS): 9.523 ns
  - Worst Hold Slack (WHS): 0.025 ns
  - Worst Pusle Width Slack (WPWS): 8.75 ns

- To znači da je **maksimalno kašnjenje kritične putanje** zapravo:

$$\mathbf{T_{kriticno} = T_{takta} - WNS = 20\ ns - 9.523\ ns\ \ ns = 10.477\ ns.}$$

- Pošto je frekvencija obrnuto proporcionalna periodu, maksimalna moguća frekvencija (teoretski maksimalna frekvencija bez prekoračenja kašnjenja) je:

$$\mathbf{f_{max} = 1/T_{kriticno} = 1/(10.477\times10^{-9}) \approx 95.45\ \ MHz}$$

9.3 Određivanje propusne moći (throughput) i kašnjenja (latency) sistema

Broj taktova potrebnih za obradu jednog piksela je 39, dok je period takta: 20 ns

Kašnjenje (latency):

$$\mathbf{Latency = (39+13) * 10{,}5\ ns = 546\ ns}$$

Dakle, vreme potrebno za obradu jednog piksela je **546 ns**.

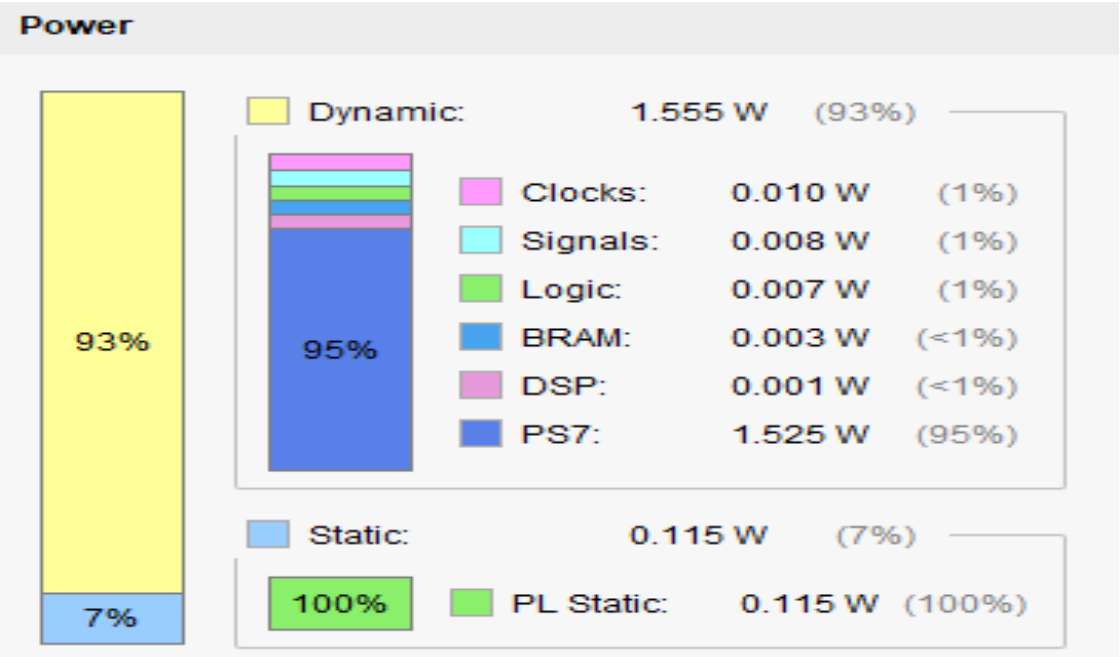
Propusna moć (throughput) predstavlja broj piksela obrađenih u sekundi:

$$\mathbf{Throughput = 1/Latency = 1/(546\times10^{-9}) \approx 1.831.500\ piksela/s}$$

Broj obrada slika u sekundi (FPS) - Za ulaznu sliku dimenzija 400*200 imamo 80.000 piskela

$$\mathbf{FPS = Throughput/80.000 = 1.831.500/80.000 \approx 22\ slika\ u\ sekundi}$$

9.4 Snaga



Slika 16, Snaga

10. Zaključak

U okviru ovog projekta uspešno je realizovana implementacija i hardverska akceleracija algoritma za detekciju objekata na osnovu boje. Na osnovu početne verzije algoritma razvijene u C jeziku, izdvojen je deo pogodniji za ubrzanje i preveden na RTL nivo korišćenjem VHDL jezika.

Kroz uklanjanje petlji i sekvencijalnu reorganizaciju algoritma, omogućeno je efikasno izvođenje na FPGA arhitekturi. Modelovan je kompletan sistem na nivou registra i transfera, uz jasan ASM dijagram, definisane komponente datapath-a i controlpath-a, i verifikovan rad pomoću simulacije.

Nakon sinteze i implementacije izvršena je detaljna analiza performansi modelovanog sistema, uključujući, utrošene resurse, kritične putanje i maksimalnu radnu frekvenciju, kašnjenje (latency) i propusnu moć (throughput).

Dobijeni rezultati su pokazali da implementacija zadovoljava funkcionalne i vremenske zahteve ciljanog algoritma. Sistem je dodatno spakovan u IP jezgro, povezano sa komponentama kao što su Zynq PS, AXI BRAM kontroler putem blok dijagrama u Vivado IP Integratoru.

Nakon integracije, sprovedena je nova analiza celokupnog integrisanog sistema, potvrđujući efikasnost u pogledu resursne optimizacije i real-time izvođenja. Na osnovu dobijenih rezultata, može se zaključiti da je akceleracija algoritma na FPGA platformi uspešno realizovana i spremna za primenu u složenijim sistemima za obradu slike u realnom vremenu.

Kako bi sistem pravilno funkcionisao slika ne sme biti veličine veće od 61x61.